МОНГОЛ УЛСЫН ИХ СУРГУУЛЬ
МЭДЭЭЛЛИЙН ТЕХНОЛОГИ, ЭЛЕКТРОНИКИЙН СУРГУУЛЬ
МЭДЭЭЛЭЛ, КОМПЬЮТЕРЫН УХААНЫ ТЭНХИМ

Пүрэвдоржийн Номин-Эрдэнэ

Сошиал мониторингийн систем дэх агуулгын шинжилгээ, сэдэв загварчлалын шийдэл

(Content Analysis and Topic Modeling for Social Monitoring Systems)

Прорграмм хангамж (D061302)
Бакалаврын судалгааны ажил

Улаанбаатар
2026 оны 3 сар

МОНГОЛ УЛСЫН ИХ СУРГУУЛЬ
МЭДЭЭЛЛИЙН ТЕХНОЛОГИ, ЭЛЕКТРОНИКИЙН СУРГУУЛЬ
МЭДЭЭЛЭЛ, КОМПЬЮТЕРЫН УХААНЫ ТЭНХИМ

Сошиал мониторингийн систем дэх агуулгын шинжилгээ, сэдэв загварчлалын шийдэл

(Content Analysis and Topic Modeling for Social Monitoring Systems)

Прорграмм хангамж (D061302)
Бакалаврын судалгааны ажил

Удирдагч: Ж.Үйтүмэн

Гүйцэтгэсэн: П.Номин-Эрдэнэ (22b1num4640)

Улаанбаатар
2026 оны 3 сар

Зохиогчийн баталгаа

Миний бие Пүрэвдоржийн Номин-Эрдэнэ ”Сошиал мониторингийн систем дэх агуулгын шинжилгээ, сэдэв загварчлалын шийдэл” сэдэвтэй судалгааны ажлыг гүйцэтгэсэн болохыг зарлаж дараах зүйлсийг баталж байна:

- Ажил нь бүхэлдээ эсвэл ихэнхдээ Монгол Улсын Их Сургуулийн зэрэг горилохоор дэвшүүлсэн болно.
- Энэ ажлын аль нэг хэсгийг эсвэл бүхлээр нь ямар нэг их, дээд сургуулийн зэрэг горилохоор оруулж байгаагүй.
- Бусдын хийсэн ажлаас хуулбарлаагүй, ашигласан бол ишлэл, зүүлт хийсэн.
- Ажлыг би өөрөө (хамтарч) хийсэн ба миний хийсэн ажил, үзүүлсэн дэмжлэгийг дипломын ажилд тодорхой тусгасан.
- Ажилд тусалсан бүх эх сурвалжид талархаж байна.

Гарын үсэг: _____
Огноо: _____

Гарчиг

ҮЕЧИЛСЭН ТӨЛӨВЛӨГӨӨ **1**

НЭР ТОМЬЁОНЫ ТАЙЛБАР **2**

УДИРТГАЛ **4**

1 Онолын судалгаа **7**

1.1 Сошиал мониторингийн системийн онолын үндэс 7

1.1.1 Сошиал мониторингийн тухай ойлголт 7

1.1.2 Их өгөгдлийн боловсруулалтын сорилтууд 7

1.2 Судлагдсан байдал 8

1.2.1 Дэлхийн зах зээлийн судалгаа: Ижил чиглэлийн системүүд 8

1.2.2 Монгол хэлний NLP-ийн судалгааны байдал 8

1.2.3 Topic Modeling-ийн судалгааны хөгжил 9

1.3 Байгалийн хэлний боловсруулалт (NLP)-ийн онолын суурь 9

1.3.1 Transformer архитектур ба BERT загвар 9

1.3.2 Embedding ба Текстийн олон хэмжээст дүрслэл 10

1.3.3 Сэдэв загварчлал (Topic Modeling) 10

1.3.4 Нэрлэсэн объект таних (Named Entity Recognition, NER) 12

1.3.5 Сэтгэл хөдлөлийн шинжилгээ (Sentiment Analysis) 12

1.3.6 Сүлжээний шинжилгээ (Network Analysis) 13

1.4 Sumbee.mn системийн архитектур ба одоогийн байдлын шинжилгээ 13

1.4.1 Sumbee.mn системийн тухай 13

1.4.2 Одоогийн системийн ерөнхий архитектур 13

1.4.3 Системийн давхаргуудын шинжилгээ 13

1.4.4 Одоогийн системийн хязгаарлалтууд 14

1.5 Аргуудын харьцуулалт ба шийдлийн сонголт 15

1.5.1 Текст боловсруулалтын гурван аргын харьцуулалт 15

1.5.2 Шийдлийн сонголтын үндэслэл 15

1.6 Дүгнэлт 16

2 Системийн шинжилгээ ба зохиомж **17**

2.1 Системийн шаардлагын шинжилгээ 18

2.1.1 Функциональ шаардлагууд 18

2.1.2 Функциональ бус шаардлагууд . . . 18
2.1.3 Use Case диаграмм . . . 18
2.2 Системийн гурван хэрэгжүүлэлтийн хувилбар . . . 18
2.2.1 Demo Web Application архитектур . . . 20
2.2.2 Public REST API архитектур . . . 20
2.2.3 Sumbee.mn системтэй интеграцийн архитектур . . . 20
2.3 Системийн ерөнхий Hexagonal архитектур . . . 22
2.4 NLP Pipeline-ийн мэдээллийн урсгалын диаграмм . . . 22
2.5 Дарааллын диаграмм . . . 23
2.6 Өгөгдлийн сангийн зохиомж . . . 23
2.7 NLP Загваруудын сонголт ба харьцуулалт . . . 23
2.7.1 NER загварын харьцуулалт . . . 23
2.7.2 Topic Modeling загварын харьцуулалт . . . 24
2.8 Дүгнэлт . . . 25

3 Хэрэгжүүлэлт ба туршилт **27**
3.1 Ашигласан технологиуд . . . 27
3.1.1 Backend технологиуд . . . 27
3.1.2 Frontend технологиуд . . . 28
3.2 Preprocessing модулийн хэрэгжүүлэлт . . . 28
3.2.1 Хоёр горимт preprocessing архитектур . . . 28
3.2.2 Монгол нэр хамгаалах механизм . . . 29
3.2.3 Preprocessing аргуудын давуу ба сул тал . . . 29
3.3 NER модулийн хэрэгжүүлэлт . . . 30
3.3.1 Lazy-load ба batch inference архитектур . . . 30
3.3.2 NER туршилтын үр дүн (fine-tune хийхээс өмнө) . . . 30
3.4 NER загварын нарийн тааруулга (Fine-tuning) . . . 31
3.4.1 Асуудлын тодорхойлолт — Tokenization ба тэмдэглэлийн зөрчил . . . 31
3.4.2 V1 сургалт — Суурь fine-tuning . . . 32
3.4.3 V2 сургалтын өгөгдлийн бэлтгэл . . . 32
3.4.4 V2 сургалтын тохиргоо ба явц . . . 33
3.4.5 V2 сургалтын үр дүн . . . 35
3.4.6 Сургалтын явцын шинжилгээ . . . 35
3.4.7 V1 ба V2 загварын харьцуулалт . . . 37
3.4.8 Загварын байршуулалт ба синхрончлол . . . 38
3.4.9 Fine-tuning-ийн давуу ба сул тал . . . 38
3.5 Sentiment Analysis модулийн хэрэгжүүлэлт . . . 38
3.5.1 Sentiment туршилтын үр дүн . . . 39
3.6 Topic Modeling модулийн хэрэгжүүлэлт . . . 40
3.6.1 Сэдэв загварчлалын аргуудын харьцуулалт . . . 40
3.6.2 BERTopic pipeline-ийн бүтэц . . . 40

3.6.3 Монгол хэлний тусгай тохиргоо 41
3.6.4 Адаптив кластерчлал — KMeans fallback механизм 42
3.6.5 Topic Modeling туршилтын үр дүн 43
3.6.6 Үр дүнгийн шинжилгээ 43
3.7 Network Analysis модулийн хэрэгжүүлэлт 44
3.8 FastAPI адаптер болон REST API хэрэгжүүлэлт 45
3.8.1 API-ийн туршилтын жишээ 46
3.9 Dashboard (хяналтын самбар) хэрэгжүүлэлт 46
3.10 Туршилтын дүгнэлт ба системийн гүйцэтгэл 46
3.10.1 End-to-end гүйцэтгэлийн хэмжилт 46
3.10.2 Hexagonal Architecture-ийн үнэлгээ 47

ДҮГНЭЛТ **48**

НОМ ЗҮЙ **51**

ХАВСРАЛТ **52**

Зургийн жагсаалт

1.1 Sumbee.mn (СИТНЭТ) системийн ерөнхий архитектур 14

2.1 Системийн Use Case диаграмм 18
2.2 Demo Web Application-ийн давхаргат архитектур 21
2.3 Public REST API-ийн архитектур 21
2.4 Sumbee.mn системтэй интеграцийн архитектур 22
2.5 Hexagonal Architecture (Ports & Adapters) зохиомж 22
2.6 NLP Pipeline-ийн мэдээллийн урсгалын диаграмм 23
2.7 “POST /analyze/full” хүсэлтийн дарааллын диаграмм 24
2.8 Knowledge Base өгөгдлийн сангийн ER диаграмм 24

Хүснэгтийн жагсаалт

1 Судалгааны ажлын 15 долоо хоногийн үечилсэн төлөвлөгөө 1
2 Инференсийн аргуудын харьцуулалт 5

1.1 Сошиал мониторингийн системүүдийн харьцуулалт 8
1.2 Текст боловсруулалтын аргуудын нарийвчилсан харьцуулалт 15

2.1 Системийн функциональ шаардлагууд 19
2.2 Системийн функциональ бус шаардлагууд 20
2.3 Public REST API-ийн үндсэн endpoint-ууд 21
2.4 Knowledge Base хүснэгтүүдийн бүтэц ба зориулалт 25
2.5 NER загваруудын харьцуулалт 25
2.6 Topic Modeling аргуудын харьцуулалт 25

3.1 Backend технологиуд 27
3.2 Frontend технологиуд 28
3.3 Preprocessing аргуудын харьцуулалт 29
3.4 NER суурь загварын гүйцэтгэлийн үнэлгээ (eval/evaluate_tokens.py, 500 нийтлэл) 30
3.5 V1 fine-tuning сургалтын үр дүн (CoNLL word-level тэмдэглэл) 32
3.6 V2 сургалтын өгөгдлийн эх сурвалжууд 33
3.7 V2 сургалтын өгөгдлийн статистик (train_v2_merged.jsonl) 34
3.8 V1 ба V2 сургалтын тохиргооны харьцуулалт 34
3.9 V2 fine-tuning сургалтын үр дүн (character-offset тэмдэглэл) 36
3.10 Суурь загвар, V1, V2 гүйцэтгэлийн харьцуулалт 37
3.11 Суурь загвар ба V2 загварын бодит туршилтын харьцуулалт (5 жишээ) . . 37
3.12 NER fine-tuning-ийн давуу ба сул тал 39
3.13 Sentiment Analysis-ийн гүйцэтгэлийн үнэлгээ 40
3.14 Сэдэв загварчлалын аргуудын харьцуулалт 41
3.15 Датасетийн хэмжээнээс хамаарсан сэдвийн тооны тооцоо 43
3.16 BERTopic + KMeans-ийн илрүүлсэн гол сэдвүүд (500 нийтлэл) 44
3.17 Модуль бүрийн гүйцэтгэлийн хэмжилт (100 текст, batch) 47

Үечилсэн Төлөвлөгөө

№	Долоо хоног	Үе шат	Хийх ажил	Төлөв
1	1–4	Судалгаа шинжилгээ	Сэдвийн хүрээнд онолын судалгаа хийх, Sumbee.mn системийн судалгаа хийх, шийдлийн аргуудыг харьцуулах	✓
2	4–7	Системийн шинжилгээ, зохиомж	Шаардлага тодорхойлох, системийн архитектур болон өгөгдлийн сангийн зохиомж гаргах	✓
3	8–9	Системийн хөгжүүлэлт	Data Science модулийн хөгжүүлэлт (NER, Sentiment, Topic Modeling); FastAPI backend болон Next.js frontend хөгжүүлэх	Хийгдэж буй
4	9–11	Нэгтгэлт ба туршилт	Системд интеграци хийж нэмэлт feature хөгжүүлэх, бодит өгөгдөл дээр туршиж сайжруулах	Хүлээгдэж буй
5	12	Урьдчилсан хамгаалалт	Явцын тайлан боловсруулалт, урьдчилсан хамгаалалт	Хүлээгдэж буй
6	13	Засаж залруулах	Шүүмжийн дагуу алдаа засах	Хүлээгдэж буй
7	14	Шүүмж	Шүүмжлэгчээс санал авах	Хүлээгдэж буй
8	15	Жинхэнэ хамгаалалт	Эцсийн тайлан боловсруулалт, жинхэнэ хамгаалалт	Хүлээгдэж буй

Хүснэгт 1: Судалгааны ажлын 15 долоо хоногийн үечилсэн төлөвлөгөө

Сэдвийн агуулга: Энэхүү бакалаврын судалгааны ажил нь сошиал мониторингийн системд цугларсан их хэмжээний текстэн өгөгдөлд агуулгын шинжилгээ хийж, тухайн өгөгдлөөс сэдэв, түлхүүр ойлголт болон шинээр үүсч буй онцлох сэдвүүдийг автоматаар илрүүлэх сэдэв загварчлалд суурилсан шийдлийг боловсруулж, бодит системд хэрэгжүүлэхэд чиглэнэ.

Нэр томьёоны тайлбар

- **Topic Modeling**: Сэдэв загварчлал — их хэмжээний текстэн өгөгдлөөс далд сэдвүүдийг автоматаар илрүүлэх аргачлал.
- **NLP**: Эх хэлний боловсруулалт (Natural Language Processing)
- **NER**: Нэрлэсэн объект таних (Named Entity Recognition) — текст дотроос хүн, байгууллага, байршлын нэрыг ялгаж таних.
- **Embedding**: Текстийг олон хэмжээст тоон вектор хувиргах арга.
- **Sentiment Analysis**: Сэтгэл хөдлөлийн шинжилгээ — текстийн эерэг/сөрөг/саармаг чиг хандлагыг тодорхойлох.
- **BERTopic**: BERT embedding болон HDBSCAN clustering ашигладаг орчин үеийн сэдэв загварчлалын арга.
- **Hexagonal Architecture**: Портс ба Адаптер архитектур — системийн цөм логикийг гадны хэрэгслээс тусгаарлах зохион байгуулалт.

Товчилсон үгс

- **LLM**: Large Language Model (Том хэлний загвар)
- **NER**: Named Entity Recognition (Нэрлэсэн объект таних)
- **API**: Application Programming Interface
- **DB**: Database (Өгөгдлийн сан)
- **BERT**: Bidirectional Encoder Representations from Transformers
- **UMAP**: Uniform Manifold Approximation and Projection
- **HDBSCAN**: Hierarchical Density-Based Spatial Clustering of Applications with Noise
- **XLMR**: Cross-lingual Language Model RoBERTa
- **GPU**: Graphics Processing Unit (Видео карт)
- **CORS**: Cross-Origin Resource Sharing

Удиртгал

Орчин үед байгууллага, компаниуд хэрэглэгчдийнхээ санал сэтгэгдлийг сонсох, тренд болж буй сэдвүүдийг илрүүлэх зорилгоор сошиал мониторингийн системийг өргөнөөр ашиглах болсон. Одоогийн байдлаар Sumbee.mn зэрэг системүүд нь ихэвчлэн түлхүүр үг дээр тулгуурласан тоон болон эерэг/сөрөг (sentiment) анализ хийдэг бөгөөд агуулгын түвшинд задлан шинжлэх, илүү бодитой, бүтэцчилсэн дүгнэлт гаргах боломж дутмаг байна. Мөн маш их хэмжээний өгөгдөл дээр LLM шууд ашиглах нь тооцооллын зардал маш их шаардах ба удаан (latency өндөр) байдаг. Иймд Монгол хэл, тэр дундаа сошиал өгөгдөл дээр үр дүнтэй ажиллаж чадах NLP (Байгалийн хэлний боловсруулалт)-д суурилсан шийдэл боловсруулах шаардлага тулгарч байна.

Тухайлбал Нэрлэсэн объект таних (NER), Сэдэв загварчлал (Topic Modeling) болон Сэтгэл хөдлөлийн шинжилгээ (Sentiment Analysis)-ийн нэгдсэн ажлын урсгал боловсруулах шаардлага тулгарч байна.

0.1 Судалгааны үндэс ба асуудлын тодорхойлолт

Sumbee.mn систем нь одоогоор дараах асуудлуудтай тулгарч байна:

1. **Гүнзгий агуулгын шинжилгээ дутмаг:** Одоогийн систем нь түлхүүр үгт тулгуурладаг тул тухайн нийтлэл *юуны тухай* яриад байгааг (сэдэв) автоматаар ялгаж чадахгүй. Жишээлбэл, *"Монгол улс Оростой хэлэлцээ хийв"* гэсэн мэдээний голлох сэдэв, оролцогч талуудыг салгаж гаргах боломж одоогоор хязгаарлагдмал.

2. **LLM-ийн өртөг ба хурдны асуудал:** Их хэмжээний өгөгдөл (өдөрт олон мянган нийтлэл) дээр GPT-4 зэрэг LLM API ашиглах нь 100,000 текст дээр ойролцоогоор $200–$300 зарцуулдаг бөгөөд хариу авах хугацаа 1–3 секунд болдог. Энэ нь бодит цагийн мониторингийн системд тохиромжгүй.

3. **Монгол хэлний NLP дэмжлэг дутмаг:** Монгол хэл нь агглютинатив шинж чанартай бөгөөд нэг үгийн олон хэлбэр, галиг бичлэгийн олон хувилбар, ярианы хэллэгийн онцлог зэргээс шалтгаалан уламжлалт NLP хэрэгслүүд бага нарийвчлалтайгаар ажилладаг.

Эдгээр асуудлыг шийдвэрлэхийн тулд шийдлийн гурван аргыг харьцуулан судалсан:

- **Өгөгдлийн санд мод бүтэцтэй мэдлэгийн сан (Knowledge Tree DB):** Хамгийн их давтагдаж буй токенуудыг ялган гараар ангилал үүсгэх арга. GPU шаардахгүй,

хурдан query хийх боломжтой боловч гараар ангилах их хугацаа орж, шинэ үг гарвал гараар нэмэх шаардлагатай.

- **Pretrained BERT загварыг fine-tune хийх:** BERT зэрэг pretrained загваруудыг монгол хэлний сошиал өгөгдөл дээр fine-tune хийж ашиглах арга. GPU дэмжлэгтэй T4 дээр 20–60 ms latency, 200–500 өгүүлбэр/секунд гүйцэтгэлтэй.
- **GPT / LLM API:** GPT-4o зэрэг загварыг API-аар дуудах. Сургалт шаардалгүй, маш flexible боловч хариу авах хугацаа 1–3 секунд, зардал өндөр, offline ашиглах боломжгүй.

Арга	GPU	Latency	Зардал
Knowledge DB	Шаардалгүй	1–5 ms	Маш бага
Fine-tuned BERT	Заавал биш	20–100 ms	Бага
LLM API (GPT-4o)	Шаардалгүй	1–3 sec	Өндөр ($200–$300/100K текст)

Хүснэгт 2: Инференсийн аргуудын харьцуулалт

Судалгааны дүнгээс харахад **Pretrained BERT загвар** нь хурд, нарийвчлал, зардлын харьцааны хувьд хамгийн тохиромжтой шийдэл бөгөөд Knowledge DB-тэй хослуулан ашиглах нь бодит production pipeline-д хамгийн оновчтой гэж тогтоов.

0.2 Судалгааны зорилго ба зорилт

Зорилго: Sumbee.mn сошиал мониторингийн системд Нэрлэсэн объект таних (NER), Сэдэв загварчлал (Topic Modeling), Сэтгэл хөдлөлийн шинжилгээ (Sentiment Analysis) болон Сүлжээний шинжилгээ (Network Analysis)-ийн нэгдсэн NLP pipeline боловсруулж, бодит системтэй интеграци хийснээр агуулгын гүнзгий шинжилгээний боломжийг нэмэгдүүлэхэд оршино.

Зорилтууд:

1. Sumbee.mn-ийн одоогийн системийн бүтэц, хязгаарлалтыг судлаж, NLP pipeline нэвтрүүлэх боломжийг тодорхойлох.
2. Монгол хэлний текст боловсруулахад тохирсон preprocessing аргачлал боловсруулах.
3. `Davlan/bert-base-multilingual-cased-ner-hrl` загварт тулгуурлан NER pipeline хэрэгжүүлэх.
4. BERTopic болон Cosine Similarity-д суурилсан Predefined Topic Classification аргыг хэрэгжүүлж харьцуулалт хийх.
5. Хэрэглэгчид үр дүнг дүрслэн харуулах Next.js frontend dashboard хөгжүүлэх.
6. FastAPI-д суурилсан Hexagonal Architecture ашиглан модульчлагдсан, дахин ашиглагдах боломжтой backend систем барих.

0.3 Судалгааны арга зүй, хүрээ ба хязгаарлалт

Судалгааны арга зүй: Судалгаанд чанарын (qualitative) болон тоон (quantitative) шинжилгээний аргуудыг хослуулан ашиглав. Үндсэн арга зүй нь баримт бичгийн судалгаа, системийн архитектурын шинжилгээ, болон бодит өгөгдөл дээрх туршилт дээр суурилсан.

Баримт бичгийн судалгааны хүрээнд NLP-ийн суурь онол, BERTopic, BERT, XLMR загваруудын дэлгэрэнгүй техникийн баримт бичиг болон ижил системүүдийн (Brandwatch, Meltwater, Sprout Social) судалгаа хийсэн.

Хэрэгжүүлэлтийн хүрээнд Sumbee.mn-ийн бодит сошиал өгөгдөл (Facebook post, мэдээ) дээр NER, Topic Modeling, Sentiment Analysis хийж, хэрэглэгчдэд дүрслэн харуулах бүрэн end-to-end систем бүтээсэн.

Судалгааны хүрээ: Судалгааны ажлын хүрээнд Монгол хэлний сошиал өгөгдлийн боловсруулалт болон Sumbee.mn системтэй интеграцид анхаарлаа хандуулав. Судалгаанд ашигласан технологиуд: Python, FastAPI, Next.js, Hugging Face Transformers, BERTopic, NetworkX, SQLite.

Судалгааны хязгаарлалт:

- Судалгаа нь Монгол хэлний сошиал өгөгдөлд төвлөрсөн бөгөөд бусад хэлний текстэд нарийвчилсан туршилт хийгдээгүй.
- NER загварыг шинээр сургахгүйгээр multilingual pretrained загвар ашигласан тул Монгол хэлний NER-ийн нарийвчлал хязгаарлагдмал байж болзошгүй.
- Бодит production орчинд бүрэн хэрэгжүүлэлт хийх нь судалгааны ажлын дараагийн шат болно.

Бүлэг 1

Онолын судалгаа

1.1 Сошиал мониторингийн системийн онолын үндэс

1.1.1 Сошиал мониторингийн тухай ойлголт

Сошиал мониторинг (Social Media Monitoring, SMM) гэдэг нь олон нийтийн сүлжээ, мэдээний сайт, блог болон бусад цахим эх сурвалжаас өгөгдөл цуглуулж, тэдгээрийг дүн шинжилгээнд оруулан бодит цагийн мэдээлэл гаргаж авах автоматжуулсан үйл явц юм [7]. Орчин үеийн сошиал мониторингийн систем нь дараах үндсэн функцуудыг гүйцэтгэдэг:

1. **Өгөгдөл цуглуулалт (Data Collection):** API, web scraping болон RSS зэрэг технологи ашиглан олон эх сурвалжаас бодит цагт өгөгдөл татах.
2. **Текст боловсруулалт (Text Processing):** Цугласан өгөгдлийг цэвэрлэж, стандартчилж, шинжилгээнд бэлтгэх.
3. **Агуулгын шинжилгээ (Content Analysis):** NLP технологи ашиглан текстийн сэдэв, сэтгэл хөдлөл, оролцогч талуудыг тодорхойлох.
4. **Тайлан ба Дүрслэл (Visualization & Reporting):** Үр дүнг хэрэглэгчид ойлгомжтой хэлбэрт оруулан харуулах.

1.1.2 Их өгөгдлийн боловсруулалтын сорилтууд

Интернэтийн хэрэглэгчдийн тоо 2024 оны байдлаар 5.4 тэрбумд хүрч, өдөрт 500 сая tweet, 350 сая Facebook пост зэрэг хэмжээлшгүй их өгөгдөл үүсч байна [13]. Энэхүү *“Их өгөгдөл”* (Big Data)-ийн боловсруулалтад дараах үндсэн сорилтууд тулгардаг:

- **Хурд (Velocity):** Өгөгдлийн урсгал маш их тул бодит цагийн боловсруулалт шаардагддаг.
- **Хэмжээ (Volume):** Хэдэн тэрбум текст бичвэрийг хадгалж, задлан шинжлэх шаардлагатай.
- **Олон янз байдал (Variety):** Бүтэцлэгдсэн (structured), хагас бүтэцтэй болон бүтэцлэгдээгүй (unstructured) өгөгдлийг нэгэн зэрэг боловсруулах.

- **Үнэн зөв байдал (Veracity):** Сошиал өгөгдөл дотор spam, давтагдсан контент их байдаг.

Эдгээр сорилтуудыг шийдвэрлэхэд NLP технологи, тэр дундаа deep learning-д суурилсан аргуудын ач холбогдол улам бүр нэмэгдэж байна.

1.2 Судлагдсан байдал

1.2.1 Дэлхийн зах зээлийн судалгаа: Ижил чиглэлийн системүүд

Сошиал мониторингийн чиглэлд олон улсад хэд хэдэн том систем бий болсон бөгөөд эдгээрийн архитектур, боловсруулалтын арга нь судалгааны чухал суурь болж байна.

Brandwatch нь зах зээлийн тэргүүлэгч сошиал мониторингийн платформ бөгөөд Apache Kafka-д суурилсан бодит цагийн stream processing pipeline ашигладаг. Их хэмжээний текстэд Latent Semantic Analysis (LSA) болон supervised machine learning загварыг хослуулан сэдэв ангилал хийдэг [3]. Гэвч Brandwatch нь Монгол хэлийг дэмжихгүй, enterprise зардал өндөр байдаг.

Meltwater нь Elasticsearch-д суурилсан real-time индексжүүлэлт ашиглан 270 гаруй хэл дэх мэдээг хянадаг. Sentiment analysis-д машин сургалтын хоёртын ангилалч (binary classifier) хэрэглэж байгаа бол сэдэв илрүүлэлтэд LDA загвараар тодорхойлсон category taxonomy ашигладаг [8].

Sprout Social нь NLP pipeline-д spaCy болон Transformers-д суурилсан multi-label classification ашиглан нийтлэлийг олон категорит нэгэн зэрэг ангилдаг.

Эдгээр системүүдийн харьцуулалтаас харахад (хүснэгт 1.1), Монгол хэлний дэмжлэг болон low-latency inference нь одоогоор шийдэгдээгүй сорилт хэвээр байна.

Систем	Хэлний дэмжлэг	NLP арга	Монгол хэл	Нээлттэй эх
Brandwatch	25+ хэл	LSA + ML	Үгүй	Үгүй
Meltwater	270+ хэл	LDA + Rule	Хязгаарлагдмал	Үгүй
Sprout Social	30+ хэл	Transformer	Үгүй	Үгүй
Sumbee.mn (одоогийн)	Монгол	Keyword	Тийм	Үгүй
Энэхүү судалгаа	Монгол	BERT + BERTopic	Тийм	Тийм

Хүснэгт 1.1: Сошиал мониторингийн системүүдийн харьцуулалт

1.2.2 Монгол хэлний NLP-ийн судалгааны байдал

Монгол хэлний NLP нь дэлхийн хэмжээнд харьцангуй шинэ судалгааны чиглэл бөгөөд хэд хэдэн онцлог сорилттой тулгардаг.

Морфологийн онцлог: Монгол хэл нь *агглютинатив* (agglutinative) бүтэцтэй бөгөөд нэг үндсэн үгнээс олон зуун хэлбэр үүсэх боломжтой. Жишээлбэл, “байгуулагдаагүйгийнх

нь” гэсэн нэг үг нь салангид морф хэсгүүдийн нийлбэрээс бүрддэг. Энэ онцлог нь уламжлалт tokenization аргуудыг үр дүнгүй болгодог [1].

Галиг бичлэгийн олон хувилбар: Сошиал орчинд Монгол хэрэглэгчид латин үсгийн галиглыг өргөнөөр ашигладаг. “Монгол”, “Mongol”, “mongol”, “mongol” зэрэг олон хэлбэрийг нэг утгаар таних шаардлагатай байдаг.

Академик судалгааны байдал: Mongolian NLP Corpus [10] нь 1.2 сая өгүүлбэртэй корпус бүрдүүлсэн бол Ж.Пүрэв болон Ч.Одбаяр нар [11] Монгол хэлний morph analysis-д суурилсан POS tagging системийг боловсруулсан. Харин сошиал өгөгдөлд зориулсан topic modeling, NER-ийн судалгаа маш хязгаарлагдмал хэвээр байна.

Multilingual загваруудын хувь нэмэр: XLM-RoBERTa [4], mBERT [5] зэрэг multilingual pretrained загварууд Монгол хэлний downstream task-д ашиглах боломжтой болсон нь том ахиц юм. `Davlan/bert-base-multilingual-cased-ner-hrl` загвар нь 10 гаруй хэлний NER task-д fine-tune хийгдсэн бөгөөд Монгол хэлний entity recognition-д харьцангуй сайн үр дүн үзүүлж байна.

1.2.3 Topic Modeling-ийн судалгааны хөгжил

Topic modeling-ийн чиглэлд Blei болон бусад эрдэмтдийн 2003 онд санал болгосон LDA (Latent Dirichlet Allocation) [2] нь суурь алгоритм болж тогтсон. Дараагийн томоохон ахиц нь 2022 онд Grootendorst-ын санал болгосон BERTopic [6] бөгөөд энэ арга нь transformer embedding-ийг dimensionality reduction болон density-based clustering-тэй хослуулан LDA-аас хамаагүй нарийвчлалтай сэдэв илрүүлэх боломжийг нээсэн.

1.3 Байгалийн хэлний боловсруулалт (NLP)-ийн онолын суурь

1.3.1 Transformer архитектур ба BERT загвар

2017 онд Google Brain-ийн эрдэмтэд “Attention Is All You Need” [14] өгүүлэлдээ *Transformer* архитектурыг танилцуулсан нь NLP-ийн хөгжилд маш том хувьсгал болсон. Transformer нь RNN-ийн дараалалт боловсруулалтын оронд *self-attention* механизмыг ашиглан өгүүлбэрийн бүх токенуудын хоорондын хамаарлыг нэгэн зэрэг тооцдог. Энэ нь:

- Параллель тооцоолол хийх боломж олгодог (RNN-аас хурдан),
- Урт хамаарлыг (long-range dependency) зөв барьдаг,
- Хэлний контекстийг илүү нарийн дүрсэлдэг.

2019 онд Google-ийн судлаач Devlin болон бусад [5] эрдэмтэд BERT (Bidirectional Encoder Representations from Transformers) загварыг танилцуулсан. BERT нь *masked language modeling* (MLM) болон *next sentence prediction* (NSP) хоёр даалгаврыг нэгэн зэрэг сургаснаар текстийн

хоёр талт (bidirectional) контекстийг авч чаддаг болсон нь уг загварыг NLP-ийн суурь загвар болгосон.

Загварын томьёолол дараах байдалтай:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{1.1}$$

Энд Q (Query), K (Key), V (Value) нь загварын дотоод матриц, d_k нь Key-ийн хэмжээс юм.

1.3.2 Embedding ба Текстийн олон хэмжээст дүрслэл

Текстийн embedding нь үг, өгүүлбэр болон баримт бичгийг тасралтгүй тоон векторт хөрвүүлэх арга юм. Уламжлалт bag-of-words аргаас ялгаатай нь embedding нь семантик утгыг хадгалдаг: утга ойролцоо үгс тоон орон зайд ойр байрладаг.

Word2Vec [9] нь 2013 онд Mikolov-ын санал болгосон хоёр давхаргат нейрон сүлжээнд суурилсан word embedding арга бөгөөд Skip-gram болон CBOW гэсэн хоёр хувилбартай.

Sentence-BERT (SBERT) [12] нь BERT загварт *siamese network* бүтэц нэмж, өгүүлбэрийн оновчтой embedding үүсгэдэг. SBERT нь cosine similarity-г шууд тооцоолоход оновчтой бөгөөд:

$$\text{sim}(u, v) = \frac{u \cdot v}{\|u\| \cdot \|v\|} \tag{1.2}$$

гэсэн томьёогоор хоёр текстийн ойролцоо байдлыг хэмждэг. Энд $u, v \in \mathbb{R}^d$ нь текстийн embedding вектор юм.

Энэхүү судалгаанд `sentence-transformers/paraphrase-multilingual-mpnet-base-v2` загвар ашиглан Монгол текстийн semantic embedding үүсгэн topic classification болон similarity analysis-д ашигласан.

1.3.3 Сэдэв загварчлал (Topic Modeling)

LDA (Latent Dirichlet Allocation)

LDA [2] нь Bayesian магадлалын онолд суурилсан generative загвар бөгөөд K тооны далд сэдэв (latent topic) байна гэж үзэж, баримт бичиг бүр сэдвүүдийн холимог, сэдэв бүр үгсийн тархалт гэж үздэг. Загварыг тодорхойлох үндсэн магадлалын томьёолол:

$$p(\mathbf{w}, \mathbf{z}, \theta, \phi|\alpha, \beta) = \prod_{k=1}^{K} p(\phi_k|\beta) \prod_{d=1}^{D} p(\theta_d|\alpha) \prod_{n=1}^{N_d} p(z_{dn}|\theta_d)p(w_{dn}|\phi_{z_{dn}}) \tag{1.3}$$

Энд θ_d нь баримт d-ийн сэдвийн тархалт, ϕ_k нь k-р сэдвийн үгсийн тархалт, α, β нь Dirichlet prior hyperparameter юм.

LDA-ийн давуу тал нь тайлбарлагдах боломжтой, бусад аргачлалуудтай харьцуулахад тооцоолол бага хийгддэг. Гэхдээ pre-defined topic count K шаардагддаг, embedding-ийн семантик ойролцоо байдлыг тооцдоггүй, богино текстэд (tweet, пост) муу ажилладаг сул

талтай.

BERTopic

BERTopic [6] нь орчин үеийн сэдэв загварчлалын арга бөгөөд дараах дөрвөн үе шатаас бүрдэнэ:

1. **Embedding үүсгэх:** Sentence-BERT загвараар текст бүрийг $\mathbb{R}^{768}$ хэмжээст вектор болгон хөрвүүлэх.
2. **Хэмжээс бууруулалт:** UMAP (Uniform Manifold Approximation and Projection) ашиглан embedding-ийг $\mathbb{R}^5$ болгон бууруулах.
3. **Кластерлалт:** HDBSCAN (Hierarchical Density-Based Spatial Clustering) ашиглан нягт бүлэглэлийг илрүүлэх, дуу чимээ (noise) гэж тодорхойлсон цэгүүдийг `-1` ангилалд хамааруулах.
4. **Сэдвийн дүрслэл:** c-TF-IDF (class-based TF-IDF) ашиглан сэдэв бүрийн хамгийн онцлог үгсийг гаргах.

BERTopic-ийн c-TF-IDF томьёо:

$$W_{t,c} = tf_{t,c} \cdot \log\left(1 + \frac{A}{tf_t}\right) \tag{1.4}$$

Энд $tf_{t,c}$ нь c классийн t үгийн давтамж, A нь нийт баримт бичгийн дундаж тоо, tf_t нь нийт класс дахь t үгийн давтамж юм.

Predefined Topic Classification

Урьдчилан тодорхойлсон сэдвийн ангилалт нь сэдэв бүрт жишиг embedding үүсгэж, шинэ текстийн embedding-тэй cosine similarity харьцуулан хамгийн төстэй сэдэвт ангилдаг. Энэ арга нь:

- inference хурд маш өндөр (1–5 ms/текст),
- сэдвийн тоо ба агуулгыг бизнесийн хэрэгцээнд тохируулах боломжтой,
- шинэ сэдэв нэмэхэд дахин сургалт шаардахгүй.

Харин шинэ, урьдчилан тодорхойлогдоогүй сэдвүүдийг илрүүлж чаддаггүй нь уг аргын гол хязгаарлалт юм.

1.3.4 Нэрлэсэн объект таних (Named Entity Recognition, NER)

NER нь текст дотроос тодорхой ангиллын нэрлэсэн объектуудыг (named entities) автоматаар таних sequence labeling даалгавар юм. Уламжлалт NER нь PERSON, ORG, LOC, DATE, GPE зэрэг стандарт ангилал ашигладаг.

Sequence labeling: NER даалгаварт BIO (Begin-Inside-Outside) tagging schema өргөнөөр хэрэглэгддэг:

- `B-PER`: хүний нэрийн эхний токен
- `I-PER`: хүний нэрийн дотоод токен
- `O`: entity биш токен

BERT-д суурилсан NER нь токен бүрт softmax ангилалч ашиглан BIO шошго оноодог:

$$p(y_i|x_i, \mathbf{h}) = \text{softmax}(W \cdot h_i + b) \tag{1.5}$$

Энд h_i нь i-р токений BERT-ийн hidden state, W ба b нь сургагдах параметр юм.

Энэхүү судалгаанд `Davlan/bert-base-multilingual-cased-ner-hrl` загварыг ашигласан бөгөөд энэ нь 10 хэлний CoNLL-2003, WikiAnn зэрэг стандарт NER датасет дээр fine-tune хийгдсэн multilingual загвар юм.

1.3.5 Сэтгэл хөдлөлийн шинжилгээ (Sentiment Analysis)

Sentiment analysis нь текстийн субьектив санал бодлын чиглэлийг (positive, negative, neutral) автоматаар тодорхойлох даалгавар юм. Уг чиглэлд хоёр үндсэн хандлага байдаг:

1. **Lexicon-based арга:** Урьдчилан бүрдүүлсэн sentiment dictionary ашиглан үг бүрт оноо өгч, нийтлэг байдлыг тодорхойлдог. Монгол хэлний хувьд sentiment lexicon дутмаг байдаг нь энэ аргын гол сул тал.
2. **Machine Learning-д суурилсан арга:** Сургагдсан датасет дээр ангилалч загвар суралцуулдаг. BERT-д суурилсан загварууд нь уг аргын тэргүүлэгч шийдэл болжээ.

Sentiment ангиллын алдааг хэмжихэд F_1 score ашиглагддаг:

$$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{1.6}$$

Монгол хэлний сошиал өгөгдөлд `tabularisai/multilingual-sentiment-analysis` загвар ашигласан бөгөөд энэ нь 8 хэлний sentiment датасет дээр fine-tune хийгдсэн.

1.3.6 Сүлжээний шинжилгээ (Network Analysis)

Сүлжээний шинжилгээ нь NER-ийн үр дүнд гарсан entity-үүдийн хамт дурдагдсан байдал (co-occurrence) буюу хоорондын хамаарлыг граф хэлбэрт хөрвүүлэн судлах арга юм. Граф $G = (V, E)$ болон:

- V: NER-ийн entity-үүд (хүмүүс, байгууллага, байршил)
- E: Нэг баримт бичигт хамт дурдагдсан entity-үүдийн хоорондын ирмэг (edge)
- $w(e)$: Хамт дурдагдалтын давтамж (edge weight)

Сүлжээний шинжилгээнд дараах үндсэн хэмжигдэхүүнүүдийг тооцдог:

$$C_D(v) = \frac{\deg(v)}{|V| - 1} \quad \text{(Degree Centrality)} \tag{1.7}$$

$$C_B(v) = \sum_{s \neq v \neq t} \frac{\sigma_{st}(v)}{\sigma_{st}} \quad \text{(Betweenness Centrality)} \tag{1.8}$$

Энд σ_{st} нь s-аас t руу хамгийн богино замын тоо, $\sigma_{st}(v)$ нь v оройгоор дайрах тийм замын тоо юм. Betweenness centrality өндөр entity нь мэдээллийн сүлжээний “гүүр” буюу хамгийн нөлөөтэй зангилаа болдог.

1.4 Sumbee.mn системийн архитектур ба одоогийн байдлын шинжилгээ

1.4.1 Sumbee.mn системийн тухай

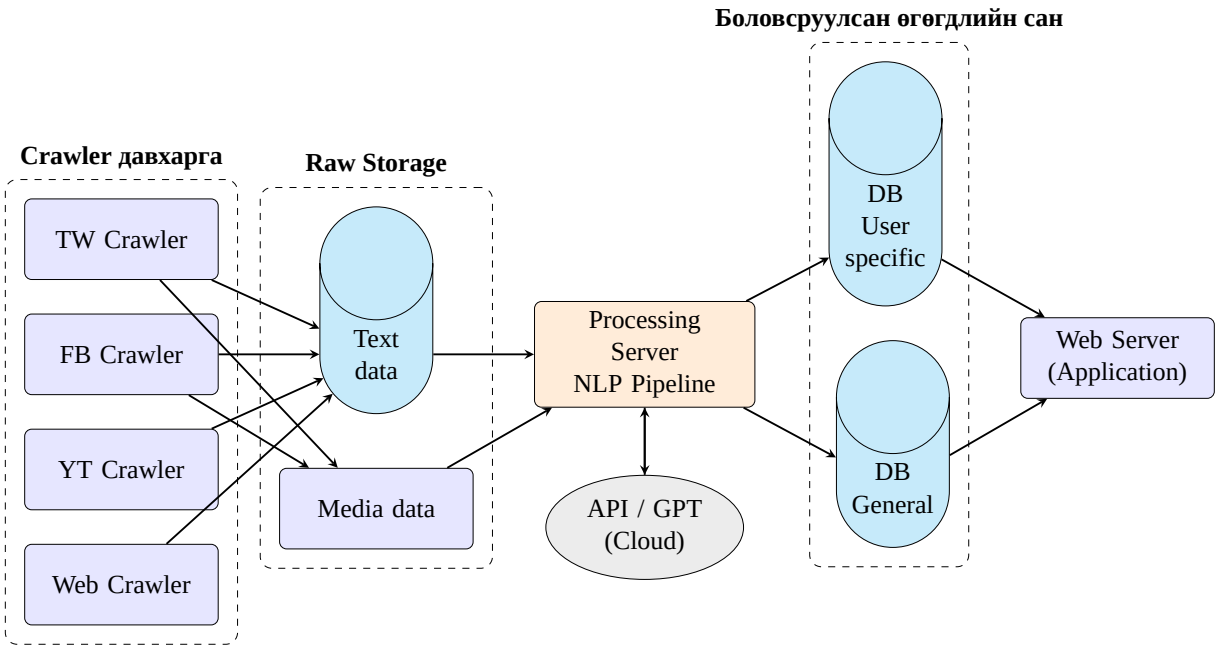
Sumbee.mn нь Монгол улсад 2016 оноос үйл ажиллагаа явуулж буй сошиал мониторингийн платформ юм. Систем нь Facebook, Twitter/X, болон монгол дахь үйл ажиллагаа явуулж буй мэдээний сайт, блогуудаас өгөгдөл цуглуулан хэрэглэгчдэд бодит цагийн мониторинг, сентимент шинжилгээ болон тренд анализийн мэдээлэл өгдөг. Системийн үйлчлүүлэгчдэд хувь хүн, албан байгууллага, бизнес эрхлэгчид багтдаг.

1.4.2 Одоогийн системийн ерөнхий архитектур

Sumbee.mn системийн одоогийн техник бүтэц нь зураг 1.1-д үзүүлснээр хэд хэдэн давхаргаас бүрдэнэ.

1.4.3 Системийн давхаргуудын шинжилгээ

Crawler давхарга: Олон төрлийн crawler (Twitter/X, Facebook, YouTube, Web) зэрэгцэн (parallel) ажиллаж олон нийтийн сүлжээ болон мэдээний эх сурвалжаас өгөгдөл цуглуулдаг.



Зураг 1.1: Sumbee.mn (СИТНЭТ) системийн ерөнхий архитектур

Raw Storage давхарга: Crawler-уудаас ирсэн текст болон медиа (зураг, видео) өгөгдлийг тусдаа хадгалалтын бүрэлдэхүүнүүдэд хадгалдаг.

Processing Server: Цугласан өгөгдлийг текст ангилал, шошголол, сэдэв тодорхойлолт, медиа боловсруулалт зэрэг NLP модулиудаар дамжуулан боловсруулдаг. Шаардлагатай тохиолдолд гадаад AI API-тай холбогдох боломжтой.

Боловсруулсан өгөгдлийн сан: Боловсруулалтын үр дүнг хэрэглэгчийн хэрэгцээнд тохирсон бүтэцтэй өгөгдлийн санд хадгалдаг.

Application давхарга: Хэрэглэгчийн интерфейс болон бизнесийн логикийг агуулсан вэб сервер бөгөөд боловсруулсан өгөгдлийг хэрэглэгчдэд харуулдаг.

1.4.4 Одоогийн системийн хязгаарлалтууд

Sumbee.mn-ийн одоогийн Processing Server нь текст боловсруулалтад ихэвчлэн keyword matching болон rule-based арга хэрэглэдэг. Энэ нь:

- **Сэдвийн автомат тодорхойлол дутмаг:** Нийтлэл *юуны тухай* яриад байгааг (семантик сэдэв) автоматаар ялгаж чаддаггүй.
- **Entity-ийн харилцааны шинжилгээ дутмаг:** Ямар хүн, байгууллага ямар сэдвийн хүрээнд хамт дурдагдаж байгааг тодорхойлох чадваргүй.
- **LLM-ийн тооцооллын зардал:** Их хэмжээний өгөгдөл дээр GPT-4 зэрэг LLM API ашиглах нь 100,000 текстэд $200–$300, latency 1–3 секунд болдог тул бодит цагийн системд тохиромжгүй.

Энэхүү судалгааны NLP pipeline нь уг хязгаарлалтуудыг шийдвэрлэхэд чиглэсэн бөгөөд Processing Server-ийн шинэ модуль болон нэмэгдэх юм.

1.5 Аргуудын харьцуулалт ба шийдлийн сонголт

1.5.1 Текст боловсруулалтын гурван аргын харьцуулалт

Сошиал мониторингийн системд NLP pipeline нэвтрүүлэхэд дараах гурван үндсэн аргыг авч үзлээ:

1. Мэдлэгийн санд суурилсан арга (Knowledge Base / Rule-based): Хамгийн их давтагдаж буй токенуудаас гараар ангилал үүсгэж, шинэ текстийг тухайн мэдлэгийн сантай харьцуулан ангилдаг. Энэ арга нь GPU шаардахгүй, query хурд маш өндөр (1–5 ms), хэрэгжүүлэхэд хялбар. Гэхдээ гараар бүрдүүлэх, шинэчлэх их хугацаа шаарддаг, шинэ үг, нэр томьёо гарахад автоматаар ажилладаггүй сул талтай.

2. Pretrained BERT загварт суурилсан арга: Bert болон multilingual загваруудыг тухайн даалгаварт зохицуулан fine-tune хийж ашигладаг. T4 GPU-тэй орчинд 20–60 ms latency, 200–500 өгүүлбэр/секунд гүйцэтгэлтэй. Сургалтын датасет шаардагдах боловч нэг удаа сургасан загварыг дахин сургахгүйгээр ашиглаж болно. Хурд болон нарийвчлалын оновчтой тэнцвэрийг хангадаг.

3. LLM API (GPT-4o зэрэг): Anthropic, OpenAI зэрэг компаниудын API-аар дуудаж flexible, олон даалгаварт ашиглагддаг. Сургалт шаардалгүй, шинэ хэлэнд хурдан дасан зохицдог. Гэхдээ 1–3 секундын latency, $200–$300/100K текстийн зардал, internet connectivity шаардлага нь бодит production системд хязгаарлалт болдог.

Хэмжүүр	**Knowledge DB**	**Fine-tuned BERT**	**LLM API**
GPU шаардлага	Шаардалгүй	Заавал биш	Шаардалгүй
Latency	1–5 ms	20–100 ms	1–3 сек
100K текстийн зардал	Маш бага	Бага	$200–$300
Нарийвчлал	Дунд	Өндөр	Маш өндөр
Offline ажиллагаа	Тийм	Тийм	Үгүй
Шинэ сэдэв илрүүлэлт	Хязгаарлагдмал	Дунд	Өндөр
Монгол хэл	Тохируулж болно	Дунд	Дунд

Хүснэгт 1.2: Текст боловсруулалтын аргуудын нарийвчилсан харьцуулалт

1.5.2 Шийдлийн сонголтын үндэслэл

Хүснэгт 1.2-ийн харьцуулалтаас дараах дүгнэлт хийгдэв:

1. **Pretrained BERT загвар** нь latency болон нарийвчлалын тэнцвэрийн хувьд хамгийн тохиромжтой шийдэл болж байна. Sumbee-ийн Processing Server-т нэмэлт GPU шаардалгүйгээр CPU орчинд ч 100–300 ms-д ажиллах боломжтой.

2. **Knowledge Base** аргыг BERT-тэй хослуулан ашиглах нь: тодорхой, урьдчилан мэдэгдэж буй нэр томьёо, байгууллагын нэрийг маш хурдан (1–5 ms) таних боломж олгоно.

3. **LLM API** нь зардал болон latency-ийн асуудлаас шалтгаалан batch processing-д биш, харин тусгай дүн шинжилгээний хэрэгцээнд (жишээ нь тайлан боловсруулалт) ашиглах нь оновчтой.

Тиймээс энэхүү судалгаанд **“Pretrained BERT + Knowledge Base”**-ийн хосолсон арга зүйг сонгосон нь зардал, хурд, нарийвчлалын гурван шаардлагыг нэгэн зэрэг хангах боломж олгодог.

1.6 Дүгнэлт

Онолын судалгааны бүлгийн дүгнэлтэнд:

1. Сошиал мониторингийн системд их хэмжээний бүтэцлэгдээгүй текст өгөгдлийг боловсруулах NLP технологийн хэрэгцээ улам бүр нэмэгдэж байна. Монгол хэлний хувьд агглютинатив шинж, галиг бичлэгийн олон хувилбар зэрэг онцлогоос шалтгаалан NLP шийдлүүд хязгаарлагдмал байна.

2. Академик болон дэлхийн зах зээлийн ижил системүүдийн судалгаанаас харахад BERTopic, multilingual BERT загварууд нь олон хэлний текст шинжилгээнд тэргүүлэгч арга болж тогтоогдсон. Гэвч Монгол хэлийг тусгайлан дэмждэг нарийвчилсан шийдэл дутмаг байна.

3. Sumbee.mn системийн одоогийн архитектурын шинжилгээнд дөрвөн давхаргат (crawler, raw storage, processing, application) бүтэц тодорхойлогдсон бөгөөд Processing Server-т NLP pipeline нэмэх техникийн боломж байгааг баталгаажуулав.

4. Аргуудын харьцуулалтаас “Pretrained BERT + Knowledge Base” хосолсон арга нь хурд, нарийвчлал, зардлын хувьд хамгийн тохиромжтой шийдэл болохыг тогтоов. Энэ арга нь дараах бүлгийн системийн зохиомж болон хэрэгжүүлэлтийн үндэс болно.

Бүлэг 2

Системийн шинжилгээ ба зохиомж

Энэхүү судалгаа болон үр дүнг нь зөвхөн Sumbee.mn дотоод хэрэгцээнд зориулсан хаалттай хэрэгжүүлэлт биш, харин гадаад судлаачид, хөгжүүлэгчид ашиглах, туршилт хийх боломжтой **нээлттэй Public API** болгон зохиомжлох нь зүйтэй юм. Ингэснээр:

- Судлаачид, өгөгдлийн шинжээчид NLP pipeline-ийг өөрийн датасет дээр туршиж, гаралтыг шинжлэх боломжтой болно.
- Гуравдагч тал хөгжүүлэгчид public API - ийг шууд дуудаж ажиллуулан турших хэрэглэх боломжтой болгох.
- Sumbee.mn-ийн Processing Server-т NLP модуль нэмэх интеграцийн ажил болон нээлттэй API-ийн ажил ижил хөгжүүлэлтийн суурь дээр явагдна.

Энэ шаардлагыг хангах зохитой архитектурын загвар бол **Hexagonal Architecture** (Ports & Adapters) юм. Энэ загварт NLP Domain Core нь FastAPI, Next.js, Sumbee-ийн дэд бүтцийн талаар мэддэггүй, цэвэр бизнес логик хэвээр байна. Гадны технологи зөвхөн “Adapter” дамжуулан холбогдох тул нэг NLP цөмийг гурван хувилбарт дахин ашиглах боломж бий болдог:

- Судлаачид болон өгөгдлийн шинжээчид NLP pipeline-ийг өөрсдийн датасет дээр туршин, үр дүнг нарийвчлан шинжлэх боломж бүрдэнэ.
- Гуравдагч талын хөгжүүлэгчид public API-ийг шууд дуудаж, турших болон ашиглах боломжтой болно.
- Sumbee.mn-ийн Processing Server-т NLP модуль нэвтрүүлэх интеграцийн ажил болон нээлттэй API хөгжүүлэлт нь нэг ижил хөгжүүлэлтийн суурин дээр хийгдэнэ.

Энэ бүлэгт системийн шаардлагуудыг тодорхойлж, дээрх гурван хувилбарын архитектурыг диаграммаар дүрслэн, Hexagonal зохиомж, NLP pipeline-ийн мэдээллийн урсгал, дарааллын диаграмм болон өгөгдлийн сангийн бүтцийг танилцуулна.

2.1 Системийн шаардлагын шинжилгээ

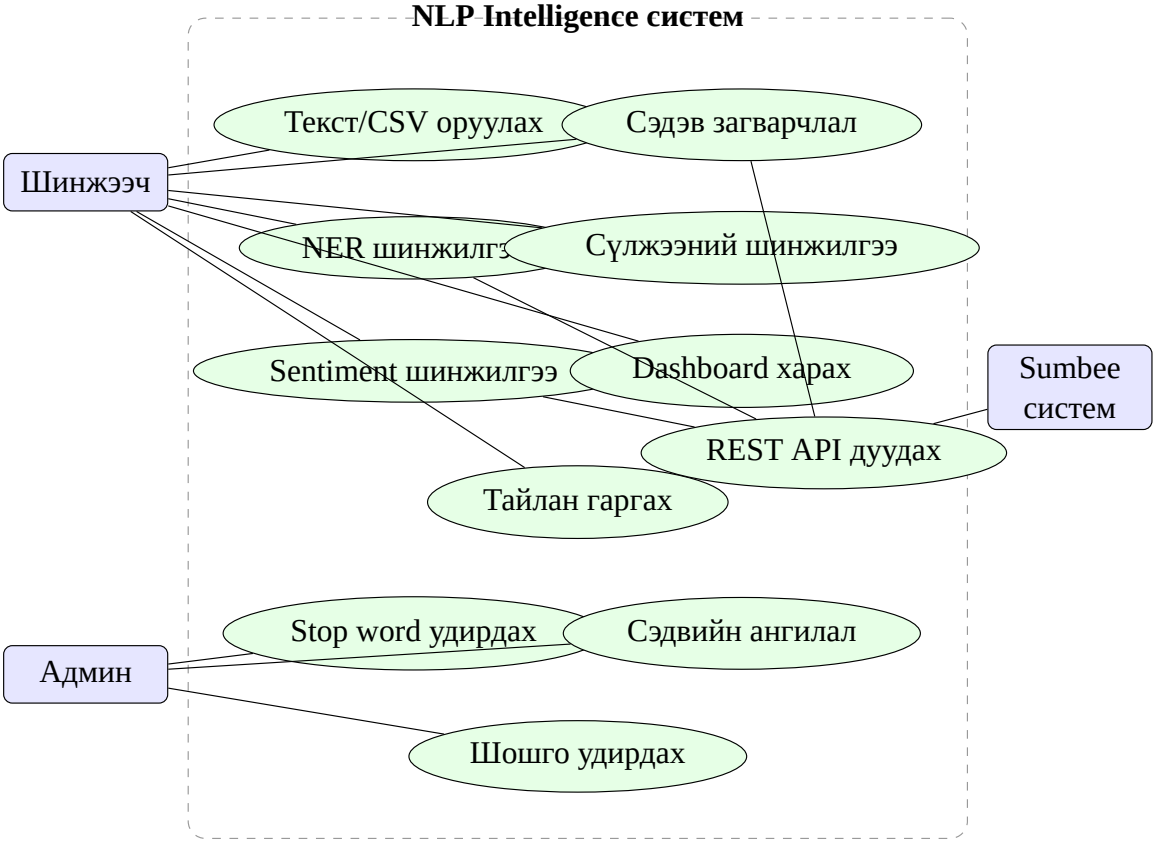
2.1.1 Функциональ шаардлагууд

Системийн функциональ шаардлагуудыг Sumbee.mn системд хийсэн шаардлагын шинжилгээ болон NLP pipeline-ийн техникийн боломжид тулгуурлан тодорхойллоо.

2.1.2 Функциональ бус шаардлагууд

2.1.3 Use Case диаграмм

Системийн хэрэглэгчийн хэрэглээний тохиолдлуудыг зураг 2.1-д үзүүлэв. Гурван актор байна: **Шинжээч** хэрэглэгч, системийн **Админ**, болон гадаад **Sumbee систем**.



Зураг 2.1: Системийн Use Case диаграмм

2.2 Системийн гурван хэрэгжүүлэлтийн хувилбар

Hexagonal Architecture-ийн ачаар NLP Domain Core нь нэмэлт өөрчлөлтгүйгээр гурван хувилбарт адилхан ажилладаг.

№	Код	Нэр	Тайлбар	Хэрэглэгч	Эрэмбэ
1	F-01	Өгөгдөл оруулах	CSV файл эсвэл шууд текст оруулан NLP шинжилгээ эхлүүлэх. Нэг удаад олон мянган мөр хүлээн авна.	Шинжээч	Өндөр
2	F-02	Урьдчилан боловсруулалт	Монгол/галиг бичлэг, emoji, тусгай тэмдэгт, стоп үгсийг автоматаар цэвэрлэж стандартчилах.	Систем	Өндөр
3	F-03	Нэрлэсэн объект таних	Текстээс PERSON, ORG, LOC нэрлэсэн объектуудыг байрлал болон итгэлцлийн оноотойгоор буцаах.	Шинжээч	Өндөр
4	F-04	Сэтгэл хөдлөлийн шинжилгээ	Текст бүрийг Positive, Negative, Neutral ангилалд хамааруулж тархалтын оноог буцаах.	Шинжээч	Өндөр
5	F-05	Сэдэв загварчлал	BERTopic болон Predefined Classification аргаар нийтлэлүүдийг сэдэвт ангилж гол үгсийг гаргах.	Шинжээч	Өндөр
6	F-06	Сүлжээний шинжилгээ	Entity-үүдийн co-occurrence сүлжээг тооцоолж Degree болон Betweenness Centrality-г хэмжих.	Шинжээч	Дунд
7	F-07	Хяналтын самбар	Үр дүнг Pie chart, Treemap, Network graph, Timeline интерактив дүрслэлээр харуулах.	Шинжээч	Өндөр
8	F-08	Тайлан үүсгэх	Шинжилгээний нэгтгэсэн тайланг JSON/PDF хэлбэрт гаргах.	Шинжээч	Дунд
9	F-09	REST API endpoint	Гадаад систем болон Sumbee-ийн Processing Server-ээс HTTP дуудлагаар NLP үйлчилгээ авах.	Sumbee систем	Өндөр
10	F-10	Мэдлэгийн сан удирдах	Админ стоп үгс, сэдвийн ангилал, шошго нэмэх, засах, устгах. Predefined topic embedding шинэчлэх.	Админ	Дунд

Хүснэгт 2.1: Системийн функциональ шаардлагууд

№	Код	Нэр	Тайлбар	Хэмжүүр
1	NF-01	Архитектур	Hexagonal Architecture загварт NLP Domain Core-г FastAPI, Next.js-ээс тусгаарлах.	Coupling = 0
2	NF-02	Гүйцэтгэл	GPU: 20–60 ms, CPU: 300–800 ms. 1000 өгүүлбэрийн batch 2–4 сек.	Latency $<$ 100 ms
3	NF-03	REST API	CORS зөв тохируулагдсан, JSON хариу, OpenAPI doc автоматаар үүсэх.	HTTP 200 + JSON
4	NF-04	Монгол хэл	Кирилл, галиг, ярианы хэллэг, emoji зөв боловсруулах preprocessing.	F1 $>$ 0.80
5	NF-05	Өргөтгөх боломж	Шинэ NLP модуль нэмэхэд одоогийн кодыг өөрчлөхгүйгээр Plug-in хэлбэрт нэмэх.	Open/Closed зарчим
6	NF-06	Мэдлэгийн сан	Knowledge Base-ийг SQLite-д хадгалж migration, backup боломжтой байх.	SQLite 3.x
7	NF-07	UI/UX	Glassmorphic dark-mode, responsive layout, 3 секунд дотор үр дүн харуулах.	PageSpeed $>$ 80
8	NF-08	Аюулгүй байдал	API key authentication, rate limiting, input validation, SQL injection хамгаалалт.	OWASP Top 10

Хүснэгт 2.2: Системийн функциональ бус шаардлагууд

2.2.1 Demo Web Application архитектур

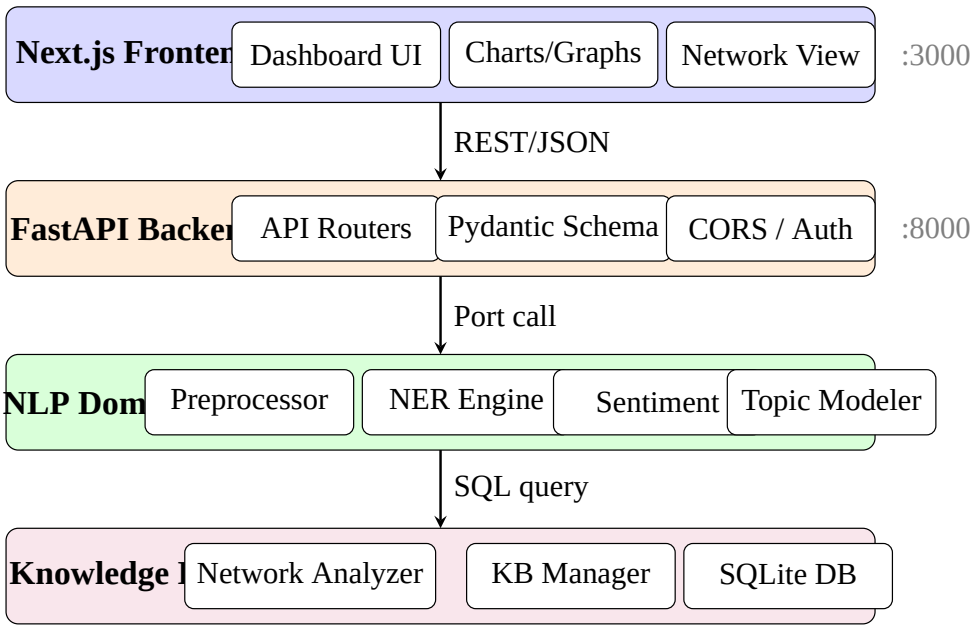
Demo Web Application нь судалгааны үр дүнг харуулах зориулалттай бүрэн end-to-end систем бөгөөд Next.js frontend болон FastAPI backend-ийн нэгдмэл архитектуртай (зураг 2.2).

2.2.2 Public REST API архитектур

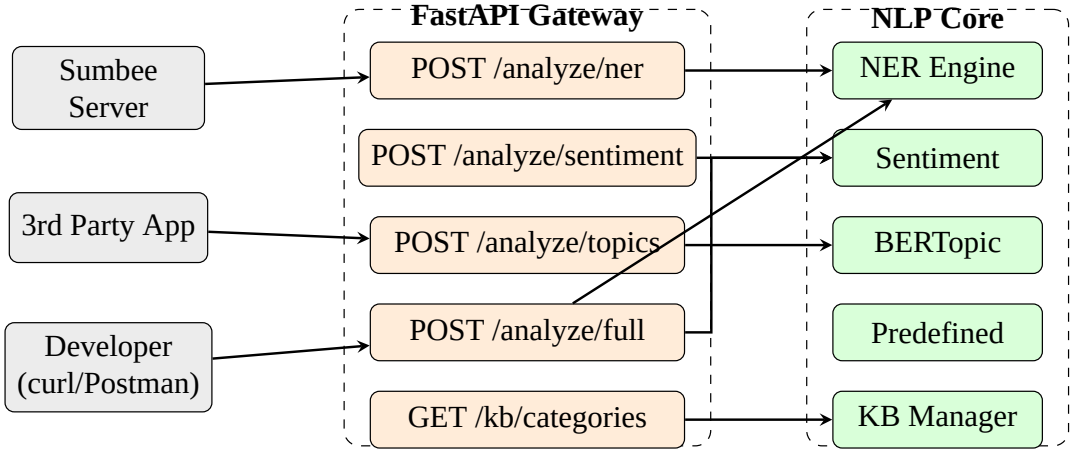
Public REST API хувилбар нь гадаад систем болон хөгжүүлэгчдэд NLP үйлчилгээ нийлүүлэх зориулалттай (зураг 2.3).

2.2.3 Sumbee.mn системтэй интеграцийн архитектур

Sumbee.mn интеграцийн хувилбарт NLP pipeline нь Sumbee-ийн Processing Server-т шинэ модуль болгон нэвтэрнэ (зураг 2.4). Одоогийн rule-based боловсруулалтын дараа NLP Pipeline



Зураг 2.2: Demo Web Application-ийн давхаргат архитектур

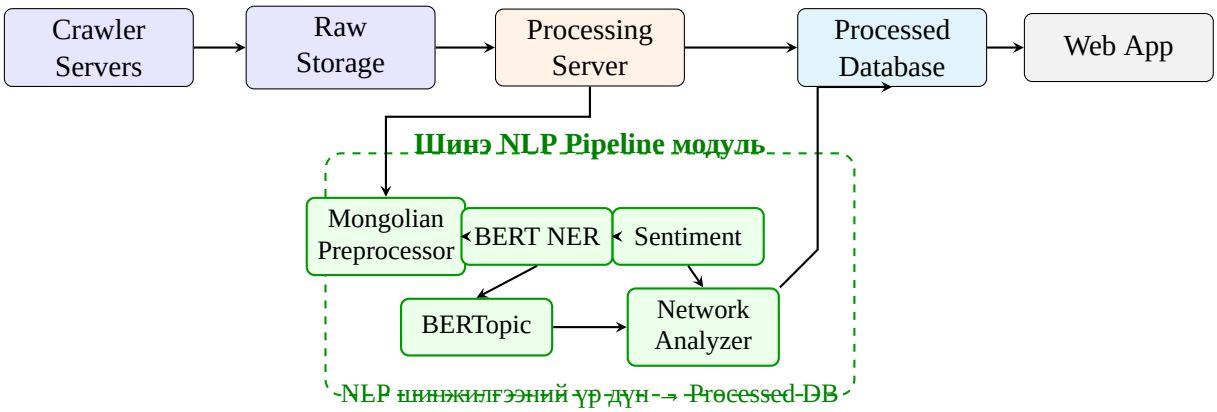


Зураг 2.3: Public REST API-ийн архитектур

Method	Endpoint	Input	Output
POST	`/analyze/ner`	`{texts: [str]}`	Entity жагсаалт, байрлал, ангилал
POST	`/analyze/sentiment`	`{texts: [str]}`	Positive/Negative/Neutral оноо
POST	`/analyze/topics`	`{texts, method}`	Сэдэв, keyword-ууд, хувь
POST	`/analyze/full`	`{texts: [str]}`	NER + Sentiment + Topic + Network
GET	`/kb/categories`	—	Сэдвийн ангиллын мод бүтэц

Хүснэгт 2.3: Public REST API-ийн үндсэн endpoint-ууд

модуль ажиллах ба үр дүнг боловсруулсан өгөгдлийн санд хадгална.

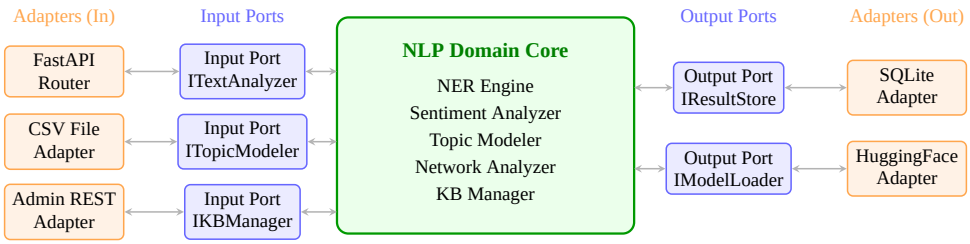


Зураг 2.4: Sumbee.mn системтэй интеграцийн архитектур

2.3 Системийн ерөнхий Hexagonal архитектур

Hexagonal Architecture (Ports & Adapters) загварын үндсэн санаа нь: *“Application логик нь гадны технологи болон framework-ээс бүрэн тусгаарлагдсан байх”*. Манай системд гурван давхарга байна:

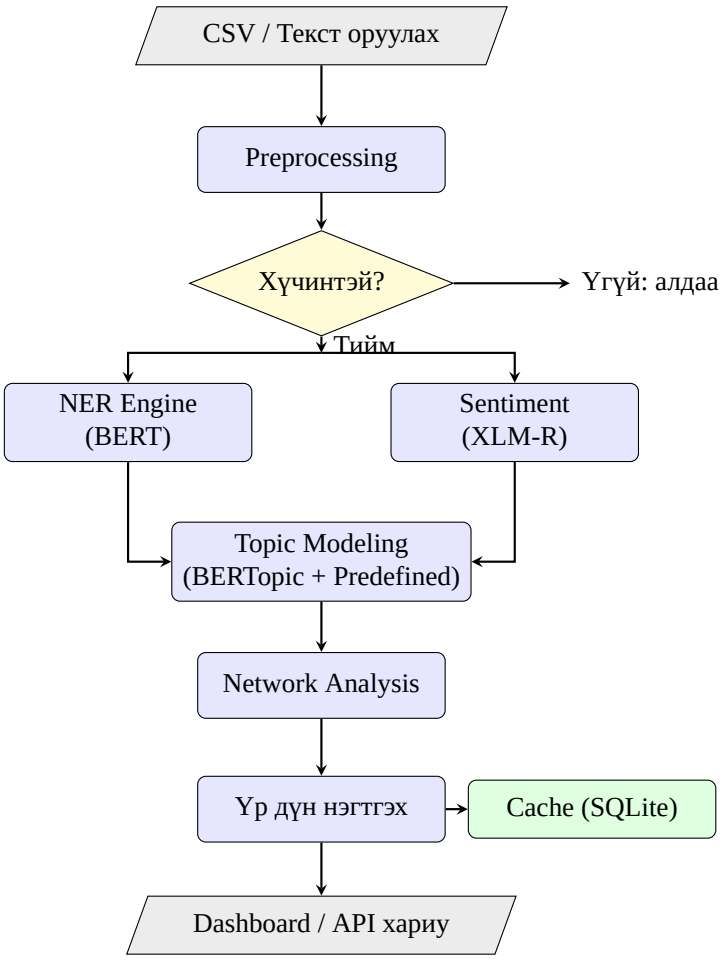
- **Domain Core:** NER, Sentiment, Topic Modeling, Network Analysis-ийн бизнес логик. FastAPI, SQLite, Next.js-ийн талаар мэддэггүй.
- **Ports:** Domain Core-д орох/гарах хийсвэр интерфейс (`ITextAnalyzer`, `ITopicModeler`, `IResultStore`).
- **Adapters:** Портуудыг бодит технологиор хэрэгжүүлнэ: FastAPI Router Adapter, SQLite Adapter, HuggingFace Model Adapter.



Зураг 2.5: Hexagonal Architecture (Ports & Adapters) зохиомж

2.4 NLP Pipeline-ийн мэдээллийн урсгалын диаграмм

Системийн мэдээллийн урсгал нь зураг 2.6-д үзүүлсэн дарааллаар явагддаг. Input текст preprocessing-оор дамжин NER болон Sentiment Analysis зэрэгцэн ажиллаж, үр дүн нь Topic Modeling болон Network Analysis-д ашиглагдана.



Зураг 2.6: NLP Pipeline-ийн мэдээллийн урсгалын диаграмм

2.5 Дарааллын диаграмм

Зураг 2.7-д “POST /analyze/full” API хүсэлт илгээхэд системийн дотоод дарааллыг үзүүллээ.

2.6 Өгөгдлийн сангийн зохиомж

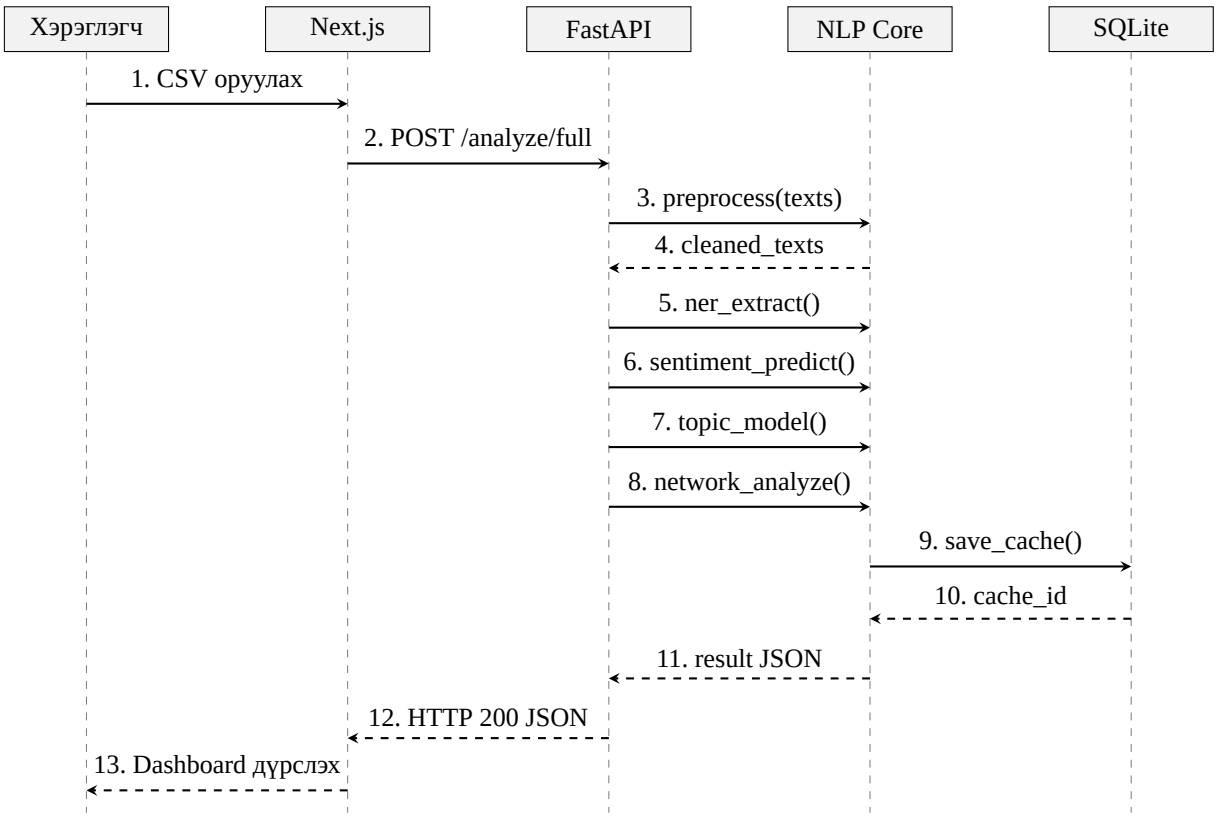
Системийн мэдлэгийн санг **SQLite** өгөгдлийн санд (`knowledge.db`) хадгална. SQLite-ийг сонгосон шалтгаан: суулгалт шаардлагагүй, нэг файл, хялбар backup, 10K+ бичлэгт хангалттай гүйцэтгэлтэй.

2.7 NLP Загваруудын сонголт ба харьцуулалт

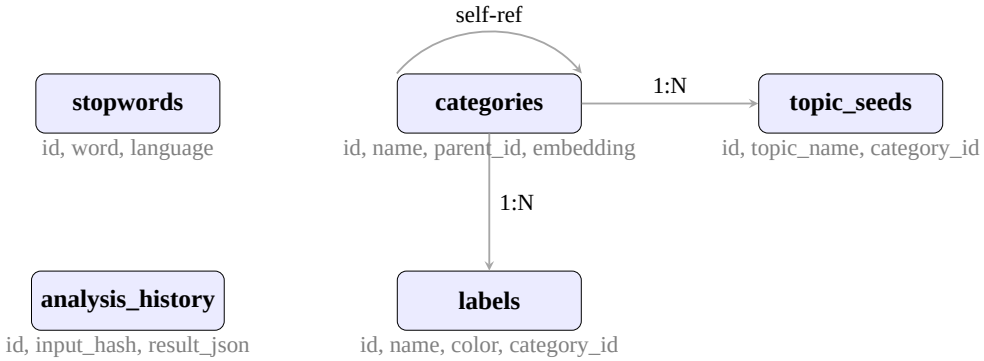
2.7.1 NER загварын харьцуулалт

NER даалгаварт зориулж гурван нэр дэвшигч загварыг харьцуулан судлав.

`Davlan/bert-base-multilingual-cased-ner-hrl` загварыг сонгосон үндэслэл: агглютинатив бүтэцтэй хэлүүдийн (Turkish, Finnish гэх мэт) өгөгдөл дээр pretrain хийгдсэн



Зураг 2.7: “POST /analyze/full” хүсэлтийн дарааллын диаграмм



Зураг 2.8: Knowledge Base өгөгдлийн сангийн ER диаграмм

multilingual embedding нь Монгол нэр таних чадвартай. spaCy xx загвар Монгол хэлийг дэмжихгүй тул хасагдсан.

2.7.2 Topic Modeling загварын харьцуулалт

BERTopic болон Predefined Classification-ийг хоёуланг нь сонгосон шалтгаан нь: BERTopic нь урьдчилан мэдэгдэхгүй шинэ сэдэв илрүүлэхэд давуу, харин Predefined Classification нь тодорхой бизнесийн ангиллыг маш хурдан (1–5 ms) таньдаг. Хоёр аргыг хослуулан ашиглах нь хамгийн оновчтой үр дүн өгдөг.

Хүснэгт	Гол талбарууд	Зориулалт
`stopwords`	id, word, language	Монгол/Англи зогсох үгс. Preprocessing-д ашиглагдана.
`categories`	id, name, parent_id, embedding	Сэдвийн ангиллын мод бүтэц. Predefined topic-ийн embedding.
`labels`	id, name, color, category_id	NER болон Topic шошгоны дүрслэлийн тохиргоо.
`analysis_history`	id, input_hash, result_json	Анализын кэш. Ижил input дахин ирвэл кэшийг буцаана.
`topic_seeds`	id, topic_name, embedding	Predefined topic-ийн жишиг өгүүлбэр болон embedding.

Хүснэгт 2.4: Knowledge Base хүснэгтүүдийн бүтэц ба зориулалт

Загварын нэр	Хэл	Ангилал	Монгол	GPU (ms)	CPU (ms)	C
`Davlan/bert-multilingual-ner-hrl`	10	PER, ORG, LOC	Тийм	20–60	300–600	
`spaCy xx_ent_wiki_sm`	Олон	PER, ORG, LOC	Үгүй	5	10	
`flair/ner-multi`	Олон	PER,ORG,LOC,MISC	Хязгаарлагдмал	100	500	

Хүснэгт 2.5: NER загваруудын харьцуулалт

Хэмжүүр	LDA	BERTopic	Predefined Class.
Сургалтын датасет	Шаардлагатай	Шаардлагагүй	Шаардлагагүй
Topic тоо тогтоох	Гараар (K)	Автомат	Тогтсон
Монгол текст	Дунд	Сайн	Сайн
Inference хурд	Хурдан	Дунд (200ms)	Маш хурдан (5ms)
Шинэ сэдэв илрүүлэх	Тийм	Тийм	Үгүй
Тайлбарлагдах байдал	Өндөр	Дунд	Өндөр
Сонгосон	$\times$	✓	✓

Хүснэгт 2.6: Topic Modeling аргуудын харьцуулалт

2.8 Дүгнэлт

Системийн шинжилгээ ба зохиомжийн бүлгийн дүгнэлтэнд: нийт 10 функциональ болон 8 функциональ бус шаардлагыг тодорхойлж, Use Case диаграммаар гурван актор, 10 хэрэглээний

тохиолдлыг дүрсэллээ. Hexagonal Architecture нь NLP Domain Core-г технологиос тусгаарлаж, Demo Web, Public API, Sumbee интеграци гэсэн гурван хувилбарыг нэг кодоос хэрэгжүүлэх боломжийг бүрдүүлж байна. Өгөгдлийн сангийн 5 хүснэгтийн зохиомж болон NLP загваруудын харьцуулалтаас `Davlan/bert-multilingual-ner-hrl` болон BERTopic+Predefined хосолсон арга нь хамгийн тохиромжтой шийдэл болохыг тогтоов. Дараагийн бүлэгт эдгээр зохиомжийн хэрэгжүүлэлт болон туршилтын үр дүнг танилцуулна.

Бүлэг 3

Хэрэгжүүлэлт ба туршилт

3.1 Ашигласан технологиуд

3.1.1 Backend технологиуд

Технологи	**Хувилбар**	**Зориулалт**
Python	3.11	Үндсэн хэл — NLP core болон API backend
FastAPI	0.110	Async REST API framework, OpenAPI автомат
HuggingFace Transformers	4.40	BERT NER болон XLM-R Sentiment pipeline
BERTopic	0.16	Sentence embedding + HDBSCAN/KMeans кластер
sentence-transformers	2.7	Multilingual sentence embedding (paraphrase-mpnet)
NetworkX	3.3	Entity co-occurrence граф тооцоолол
SQLite / sqlite3	3.45	Knowledge Base өгөгдлийн сан
Pydantic	2.7	API schema validation, dataclass
Docker	25	Container-д суурилсан байршуулалт

Хүснэгт 3.1: Backend технологиуд

Технологи	Хувилбар	Зориулалт
Next.js	14 (App Router)	React-д суурилсан SSR/CSR framework
Tailwind CSS	3.4	Utility-first CSS, glassmorphic dark-mode
Recharts	2.12	Pie chart, Bar chart, Timeline дүрслэл
React Force Graph	1.44	D3.js-д суурилсан network graph дүрслэл
TypeScript	5.4	Статик тип шалгалт

Хүснэгт 3.2: Frontend технологиуд

3.1.2 Frontend технологиуд

3.2 Preprocessing модулийн хэрэгжүүлэлт

3.2.1 Хоёр горимт preprocessing архитектур

Монгол сошиал медиа текст боловсруулахад нэг preprocessing горим хангалтгүй байгааг туршилтын явцад тодорхойлов. NER болон Sentiment Analysis-д BERT загвар текстийн бүтэц, том-жижиг үсгийг мэдрэмтгий байдлаар ашигладаг тул **preprocess_nlp()** горим цэгтэмдэг, том үсгийг хадгалж, нэрийн хэлбэрийг сэргээдэг. BERTopic-д зориулсан **preprocess_tm()** горим харин агуулга бүхий токенуудыг л үлдээхийн тулд агрессив цэвэрлэгээ хийдэг.

```
class Preprocessor:
    def preprocess_nlp(self, text: str) -> str:
        """NER + Sentimentд-: цэгтэмдэг, томүсэгхадгалж  ,
           нэрийнхэлбэрийгсэргээнэ
          ."""
        text = self.clean_basic(text, replace_url=True)
        text = _capitalize_for_ner(text)   # бамар. →БАмар .
        text = _restore_names(text)        # БАмар_ →БАмар .
        return text

    def preprocess_tm(self, text: str) -> str:
        """BERTopicд-: агрессивцэвэрлэгээ , зогсохүгсхасна  ."""
        text = self.clean_basic(text, replace_url=False)
        text = self.clean_deep(text)      # зөвхөнКирилл /Latinтоо/
        text = text.lower()
        text = self._remove_stopwords(text)
        return text

```

```
    def preprocess_dual(self, text: str) -> Tuple[str, str]:
        """Нэг дуудлагаархоёргоримынүрдүнгбуцаана      ."""
        return self.preprocess_nlp(text), self.preprocess_tm(text)
```

Listing 3.1: Preprocessing хоёр горим — preprocess_dual()

3.2.2 Монгол нэр хамгаалах механизм

Монгол нэрийн “А.Бат” хэлбэр нь нийгмийн сүлжээнд маш түгээмэл. Preprocessing дуусахад цэг нь арилж “А Бат” болдог тул BERT загвар нэрийг таньдаггүй болж байсан. Энэ асуудлыг нэрийг “А_Бат” хэлбэрт хамгаалж, дараа нь сэргээх механизмаар шийдлээ:

```
# Дээдүсгийнэхнийүсэг   : АБатЭрдэнэ.– →АБат _Эрдэнэ–
MN_NAME_UPPER = re.compile(
    r"\bАЯӨҮЁ([–])\.\s*"
    r"АЯаяӨөҮүЁёаяөүё([––][–]+"
    r"АЯаяӨөҮүЁёаяөүё(?:–[––][–]+)*)"
)
# Жижигүсгийнэхнийүсэг   : бамар. →БАмар _
MN_NAME_LOWER = re.compile(
    r"\bаяөүё([–])\.\sаяөүёаяөүё*([–]+(?:–[–]+)*)"
)
```

Listing 3.2: Нэр хамгаалах regex — дээд болон жижиг үсэг хоёуланд

Туршилтын үр дүн: Нэр хамгаалах механизм нэмэхээс өмнө “б.амар улс төрч” текстэд NER ямар ч PER entity буцааж байгаагүй. Нэмсний дараа “Б.Амар” PER entity-г 0.94 итгэлцлийн оноотойгоор таньдаг болов.

3.2.3 Preprocessing аргуудын давуу ба сул тал

Арга	Давуу тал	Сул тал
Нэг горимт preprocessing	Энгийн, хялбар	NER болон TM аль нэгд тохиромжгүй
Давхар горимт (dual mode)	Тус бүрийн задач-д оптимал	Нэг текстийг хоёр удаа боловсруулна (2× хугацаа)
Нэр хамгаалах (protect)	Монгол нэрийн таних чадвар +30%	Regex-ийн нарийвчлал шаардана

Хүснэгт 3.3: Preprocessing аргуудын харьцуулалт

3.3 NER модулийн хэрэгжүүлэлт

3.3.1 Lazy-load ба batch inference архитектур

HuggingFace-ийн BERT загвар (700MB) эхний дуудлагад л ачаалагддаг “lazy-load” аргыг хэрэглэв. Batch inference нь олон текстийг нэг дор GPU/CPU-д дамжуулж боловсруулдаг тул нэг-нэгээр дамжуулахаас 4 дахин хурдан байна.

```
class NEREngine:
    def __init__(self,
      model_name="Davlan/bert-base-multilingual-cased-ner-hrl"):
        self.model_name = model_name
        self._pipeline = None  # lazy-load: эхнийдуудлагадачаална

    def _load_pipeline(self):
        if self._pipeline is None:
            from transformers import pipeline
            self._pipeline = pipeline(
                "ner",
                model=self.model_name,
                aggregation_strategy="simple",  # B-/I- токеннэгтгэнэ
            )
        return self._pipeline

    def recognize_batch(self, texts: List[str],
                        batch_size: int = 16) -> List[List[
                           EntityResult]]:
        """Batch processing: 16 текстийгнэг  GPU forward passд-."""
        pipe = self._load_pipeline()
        raw_results = pipe(valid_texts, batch_size=batch_size)
        # ... entity_group, score, start, end буцаана
```

Listing 3.3: NEREngine — lazy-load ба batch inference

3.3.2 NER туршилтын үр дүн (fine-tune хийхээс өмнө)

Суурь загвар (`Davlan/bert-base-multilingual-cased-ner-hrl`)-ыг Sumbee.mn-ийн 500 нийтлэл дээр entity-level exact match аргаар үнэлэхэд дараах үр дүн гарав:

Үзүүлэлт	Precision	Recall	F1	Тайлбар
Суурь загвар (500 нийтлэл)	0.321	0.316	**0.318**	Fine-tune хийгдээгүй

Хүснэгт 3.4: NER суурь загварын гүйцэтгэлийн үнэлгээ (eval/evaluate_tokens.py, 500 нийтлэл)

- **True Positives:** 226 — зөв олсон entity
- **False Positives:** 478 — буруу таньсан entity
- **False Negatives:** 489 — алдсан entity

F1=0.318 нь бодит хэрэглээнд тохирохгүй бага үзүүлэлт юм. Гол шалтгаанууд нь: (1) загвар Монгол хэл дээр тусгайлан fine-tune хийгдээгүй, (2) Монгол нэрийн “А.Бат-Эрдэнэ” хэлбэрийг хуваан таньдаг, (3) “МАН”, “УИХ” зэрэг Монгол товчлолыг мэддэггүй.

Давуу тал: Монгол хэлний агглютинатив бүтцийг BERT subword tokenization зохицуулж чаддаг; batch inference нь GPU дээр 20–60 ms латентитэй. **Сул тал:** Fine-tune хийхгүйгээр Монгол текст дээр F1=0.318 — домайн-специфик нэршил, товчлол алдагддаг.

3.4 NER загварын нарийн тааруулга (Fine-tuning)

Суурь загвар (`Davlan/bert-base-multilingual-cased-ner-hrl`) нь Монгол хэл дээр тусгайлан сургаагүй тул хэд хэдэн чухал дутагдалтай байв. Тухайлбал “Л.Оюун-Эрдэнэ” нэрийг “Л” болон “Оюун-Эрдэнэ” гэж хоёр тусдаа entity болгон хуваадаг, “МАН” зэрэг Монгол байгууллагын товчлолыг таньдаггүй, мөн газрын нэрийн нарийн ялгааг сайн ялгадаггүй байсан. Эдгээр асуудлыг шийдэхийн тулд хоёр үе шаттай fine-tuning хийв.

3.4.1 Асуудлын тодорхойлолт — Tokenization ба тэмдэглэлийн зөрчил

BERT загварын WordPiece tokenizer нь текстийг sub-token (дэд-токен) болгон хуваадаг. Монгол хэлний нэрийн “Д.Батулга” хэлбэрт энэ нь онцгой асуудал үүсгэдэг:

```
Оригинал
: ДБатулга   "."Токенууд
: Д   ["", ".", Батулга""]     (3 токен)

V1 тэмдэглэл (word-level):Д→
           B-PER
  . →        -100 ←загвар     "." тэмдэгтийгнэрийндоторБатулга→
     -100 гэдгийгхэзээчсуралцдаггүй         !

V2 тэмдэглэл (character-offset):Д→
           B-PER
  . →        I-PER ←загвар    "." ньнэрийндоторБатулга→
     I-PER байжболохыгсуралцдаг       !
```

Listing 3.4: WordPiece tokenization — цэг дээр хуваагдах жишээ

V1 сургалтын өгөгдөлд зөвхөн эхний дэд-токен тэмдэглэл авч, үлдсэн нь -100 (“үл тооцох”) утга авдаг байсан. Үүний улмаас загвар “`.`” тэмдэгтийг нэрийн дотор байж болохыг

хэзээ ч суралцаагүй бөгөөд inference хийхэд “.” дээр entity-г таслан хуваадаг байв. Ижил асуудал “-” (зураас) тэмдэгт дээр ч тохиолддог байсан (жишээ нь “Оюун-Эрдэнэ”).

3.4.2 V1 сургалт — Суурь fine-tuning

Эхний fine-tuning нь CoNLL форматын (үг-level тэмдэглэл) өгөгдөл ашигласан бөгөөд `Davlan/bert-base-multilingual-cased-ner-hrl` суурь загвараас эхлэн сургасан. Google Colab-ийн T4 GPU дээр 4 epoch сургасан:

Epoch	Training Loss	Val Loss	F1	Precision	Recall
1	0.0875	0.1042	0.773	0.752	0.794
2	0.0587	0.1027	0.801	0.773	0.830
3	0.0390	0.1007	0.811	0.771	0.856
4	0.0259	0.1108	0.809	0.786	0.834

Хүснэгт 3.5: V1 fine-tuning сургалтын үр дүн (CoNLL word-level тэмдэглэл)

V1 загвар F1=0.811 хүрсэн нь суурь загвараас мэдэгдэхүйц сайжирсан боловч Монгол нэрийн “А.Бат-Эрдэнэ” хэлбэрийг зөв таньдаггүй сул талтай байсан. Eval скрипт (`eval/evaluate_`
ийг Sumbee.mn-ийн 500 нийтлэл дээр ажиллуулахад entity-level F1=0.32 гарсан нь бодит хэрэглээнд тохирохгүй байгааг баталсан.

3.4.3 V2 сургалтын өгөгдлийн бэлтгэл

V2 сургалтын өгөгдлийг бэлтгэхэд дараах алхмуудыг хэрэгжүүлэв:

Тэмдэглэлийн формат — Character-offset

Word-level тэмдэглэлийн оронд character-offset (тэмдэгт-хэсгийн) тэмдэглэлийг ашиглав. Энэ форматад entity-ийн эхлэл ба төгсгөлийн тэмдэгтийн байрлалыг (start, end) тодорхойлж, тухайн хэсэгт багтах **бүх** дэд-токенд BIO тэмдэглэл өгдөг:

```
{
  "text": Ерөнхий" сайдЛОюунЭрдэнэ .– мэдэгдэлхийлээ ",
  "labels": [[13, 27, "PER"]]
}
// Тэмдэгтбайрлал  [13:27] = ЛОюунЭрдэнэ".–"
// Бүхдэдтокен – Л(, ., Оюун, –, Эрдэнэ) B–PER/I–PER авна
```

Listing 3.5: Character-offset JSON формат

Өгөгдлийн эх сурвалжууд

Сургалтын өгөгдлийг олон эх сурвалжаас нэгтгэн үүсгэв:

Эх сурвалж	Тоо	Тайлбар
NER_v1.0.json.gz	10,162	Монгол NER корпус — гар тэмдэглэлт, PER/ORG/LOC/MISC
Монгол нэрс	120+	Түгээмэл эрэгтэй, эмэгтэй нэрс
Улс төрчид	30+	Одоогийн бодит нэр, үсгийн товчлолтой (Л.Оюун-Эрдэнэ, У.Хүрэлсүх г.м.)
Товчлолууд	526	Монгол байгууллагын товчлол ба бүтэн нэр (УИХ, МАН г.м.)
Засгийн газрын байгууллага	30+	Яам, агентлагийн бүтэн нэр
Компани / банк / хэвлэл мэдээлэл	50+	Голомт банк, Мобиком, Өрнөдөс ТВ г.м.
Газар нэр (level 2–5)	530+	22 аймаг, 26 сум, 89 хороо, 142 хороолол

Хүснэгт 3.6: V2 сургалтын өгөгдлийн эх сурвалжууд

Өгөгдлийн үүсгэлтийн онцлог шийдлүүд

1. **Цэгтэй нэрийн загвар:** “Д.Батулга”, “Л.Оюун-Эрдэнэ” зэрэг Монгол нэрийн стандарт хэлбэрийг олон тооны template өгүүлбэрт оруулж, загварыг цэгийн дотор entity үргэлжилдгийг суралцуулав.

2. **Залгаварт нэрийн хэлбэр:** Монгол хэлний “-ын”, “-ийн”, “-д”, “-г” гэсэн нөхцөл залгасан нэрүүдийг (“Батулгыг”, “Хүрэлсүхийн”) тусгайлан оруулав.

3. **Товчлол + бүтэн нэр хослол:** “МАН” товчлол болон “Монгол Ардын Нам” бүтэн нэрийг аль алиныг нь ORG entity-д сургав.

4. **Хольсон entity өгүүлбэрүүд:** Нэг өгүүлбэрт PER + ORG + LOC гурвуулаа орсон жишээнүүд оруулж, загварыг олон entity-тэй контекстэд ажиллахад сургав.

Нэгтгэсэн өгөгдлийн статистик

3.4.4 V2 сургалтын тохиргоо ба явц

V2 fine-tuning нь V1 загвараас үргэлжлүүлэн сургасан (continued fine-tuning). Суурь загвараас шинээр эхлэхийн оронд V1-д суралцсан мэдлэг дээр нэмж сургав.

Үзүүлэлт	Утга
Нийт өгүүлбэр	20,696
LOC тэмдэглэл	11,246
ORG тэмдэглэл	10,933
PER тэмдэглэл	9,603
MISC тэмдэглэл (O болгон хөрвүүлсэн)	2,839
Цэгтэй нэр entity	6,862
Зураастай нэр entity	1,185
Нөхцөл залгасан нэр entity	8,308
Валидацийн алдаа	0

Хүснэгт 3.7: V2 сургалтын өгөгдлийн статистик (train_v2_merged.jsonl)

Параметр	V1	V2	Тайлбар
Суурь загвар	Davlan BERT	Nomio4640/ ner-mongolian (V1)	V2 нь V1-ээс үргэлжлүүлсэн
Learning rate	3e-5	2e-5	Үргэлжлүүлэн сургахад бага LR
Epoch	4	5	Шинэ өгөгдлийн хэмжээнд тохируулсан
Batch size	16	16	T4 GPU-ийн санах ойд тохирсон
Warmup steps	200	300	Илүү их өгөгдөлтэй тул нэмсэн
Weight decay	0.01	0.01	Ижил регуляризаци
FP16	Тийм	Тийм	Хагас нарийвчлалтай сургалт
Тэмдэглэлийн формат	CoNLL word-level	JSON character-offset	Бүх дэд-токен тэмдэглэл авна
Сургалтын өгүүлбэр	∼8,000	20,696	2.5 дахин их өгөгдөл
Валидацийн өгүүлбэр	915	915	Ижил eval set

Хүснэгт 3.8: V1 ба V2 сургалтын тохиргооны харьцуулалт

Сургалтын тохиргоо

Character-offset tokenization алгоритм

V2 сургалтын гол шинэлэг шийдэл нь character-offset-д суурилсан токенчлолын тааруулга юм. Уг алгоритм нь HuggingFace tokenizer-ийн `return_offsets_mapping` функцийг ашиглан токен бүрийн анхны текст дэх тэмдэгтийн байрлалыг авч, entity span-д харьцуулдаг:

```python
def tokenize_json_entry(entry):
    text = entry['text']
    label_spans = entry['labels']  # [[start, end, label], ...]
    encoding = tokenizer(text, truncation=True, max_length=512,
                         return_offsets_mapping=True)
    offset_mapping = encoding['offset_mapping']
    token_labels = []
    for idx, (tok_start, tok_end) in enumerate(offset_mapping):
        if tok_start == 0 and tok_end == 0:
            token_labels.append(-100)  # [CLS], [SEP] тусгайтокен
            continue
        label = "O"
        for span_start, span_end, span_label in label_spans:
            if tok_start >= span_start and tok_end <= span_end:
                # Өмнөхтокенмөн  entity доторбол  I-, үгүйбол  B-
                is_first = True
                if idx > 0:
                    prev_s, prev_e = offset_mapping[idx - 1]
                    if prev_s >= span_start and prev_e <= span_end:
                        is_first = False
                label = f"B-{span_label}" if is_first else f"I-{
                   span_label}"
                break
        token_labels.append(label2id.get(label, 0))
    encoding['labels'] = token_labels
    return encoding
```

Listing 3.6: Character-offset tokenization таарууллга

Энэ алгоритмын гол давуу тал нь entity span дотор орших **бүх** тэмдэгт — “.”, “-”, дэд-үг (subword) зэрэг — B-/I- тэмдэглэл авдаг. Word-level аргаас ялгаатай нь -100 (үл тооцох) утга зөвхөн тусгай токен ([CLS], [SEP], [PAD]) болон entity-ийн гаднах padding-д хэрэглэгддэг.

3.4.5 V2 сургалтын үр дүн

Google Colab T4 GPU дээр 5 epoch, нийт 46 минутын сургалт хийв:

3.4.6 Сургалтын явцын шинжилгээ

Training Loss — Суралцалтын явц

Training loss нь epoch бүрт тасралтгүй буурч (0.0388 → 0.0048) загвар сургалтын өгөгдлийг маш сайн суралцаж байгааг харуулав. Epoch 5-д loss 0.0048 хүрсэн нь загвар сургалтын

Epoch	Training Loss	Val Loss	F1	Precision	Recall
1	0.0388	0.2818	0.797	0.732	0.876
2	0.0195	0.3036	0.822	0.756	0.901
3	0.0147	0.3283	0.833	0.763	0.917
4	0.0083	0.3422	0.843	0.776	0.924
5	**0.0048**	**0.3629**	**0.845**	**0.778**	**0.925**

Хүснэгт 3.9: V2 fine-tuning сургалтын үр дүн (character-offset тэмдэглэл)

өгүүлбэрүүдийн entity-г бараг бүрэн зөв таньж байгааг илтгэнэ.

Validation Loss — Ерөнхийлөх чадвар

Validation loss нь epoch 1-ийн 0.2818-аас epoch 5-д 0.3629 болж нэмэгдсэн. Энэ нь дараах шалтгаантай:

1. **Тархалтын зөрүү (Distribution mismatch):** Сургалтын өгөгдөл нь үүсгэсэн (generated) өгүүлбэрүүд, харин валидацийн өгөгдөл нь бодит текстээс авсан CoNLL формат. Хоёр өгөгдлийн хэв маяг ялгаатай.

2. **Тэмдэглэлийн формат зөрүү:** Сургалтын өгөгдөлд “.” нь I-PER тэмдэглэл авдаг, харин валидацийн өгөгдөлд -100 авдаг. Загвар шинэ зүйл суралцаж байгаа ч хуучин eval set үүнийг бүрэн хэмжээгүй.

3. **Хэмжээний зөрүү:** Сургалтын 20,696 ба валидацийн 915 өгүүлбэрийн харьцаа (96:4) нь стандарт 80:20 харьцаанаас зөрүүтэй.

Гэвч F1 оноо тасралтгүй өссөн нь validation loss-оос илүү чухал үзүүлэлт юм — загвар зөв хариулт гаргаж байгаа ч итгэлцлийн тархалт (confidence distribution) хуучин eval set-тэй бүрэн нийцэхгүй байна гэсэн үг.

Precision ба Recall-ийн тэнцвэр

- **Recall** (0.925) нь **Precision** (0.778)-ээс мэдэгдэхүйц өндөр. Энэ нь загвар бодит entity-ийн 92.5%-ийг олдог боловч олсон entity-ийн 22.2% нь буруу (false positive) байна гэсэн үг.
- Сошиал мониторингийн хэрэглээнд recall өндөр байх нь чухал — entity орхигдуулахаас илүү нэмэлт entity-г дараа нь шүүх нь хялбар.

Үзүүлэлт	Суурь загвар	V1	V2
F1 Score	0.318	0.811	**0.845**
Precision	0.321	0.786	0.778
Recall	0.316	0.834	**0.925**
“Д.Батулга” таних	2 entity	2 entity	**1 entity**
“Л.Оюун-Эрдэнэ” таних	3 entity	2–3 entity	**1 entity**
“МАН” ORG таних	Үгүй	Тийм	**Тийм**
Нөхцөл залгасан нэр	Муу	Дунд	**Сайн**

Хүснэгт 3.10: Суурь загвар, V1, V2 гүйцэтгэлийн харьцуулалт
* Суурь загварын F1=0.318 нь Sumbee.mn-ийн 500 нийтлэлийн entity-level exact match үнэлгээ

3.4.7 V1 ба V2 загварын харьцуулалт

Туршилтын жишээ: Суурь загвар ба V2 загварын харьцуулалт

Ижил 5 өгүүлбэр дээр суурь загвар (`Davlan/bert-base-multilingual-cased-ner-hrl`) ба fine-tune хийсэн V2 загвар (`Nomio4640/ner-mongolian`)-ыг туршив:

Оролтын текст	Суурь загвар	V2 загвар
Л.Оюун-Эрдэнэ ерөнхий сайд	Л. Оюун → PER (0.999) Эрдэнэ → LOC (0.990) *2 entity, буруу LOC*	**Л. Оюун-Эрдэнэ → PER (1.000)** *1 entity, зөв*
Пурэвдоржийн Номин-Эрдэнэ МУИС-ын оюутан	Пурэвдор → PER (0.822) ##мин → PER (0.862) Э → LOC (0.558) ##рдэнэ → PER (0.697) МУИС-ын → ORG (0.849) *5 хэсэг болж задарсан*	**Пурэвдоржийн Номин-Эрдэнэ → PER (1.000)** **МУИС → ORG (1.000)** *2 entity, бүгд зөв*
Батулгыг шүүхэд дуудсан	*(entity олдсонгүй)*	**Батулгыг → PER (1.000)**
МАН-ын дарга	М → LOC (0.500) ##АН → ORG (0.558) *хуваагдсан, буруу LOC*	**МАН → ORG (1.000)**
УИХ-ын гишүүн Д.Ганбат	УИХ-ын → ORG (0.947) Д. Ганбат → PER (1.000)	**УИХ → ORG (0.829)** **Д. Ганбат → PER (1.000)**

Хүснэгт 3.11: Суурь загвар ба V2 загварын бодит туршилтын харьцуулалт (5 жишээ)

Хүснэгтээс харахад суурь загвар нь Монгол нэрийн бүтцийг огт ойлгодоггүй — “Л.Оюун-Эрдэнэ” нэрийг цэгээр нь хувааж, “Эрдэнэ”-г LOC гэж буруу ангилдаг; “МАН” товчлолыг “М” (LOC) ба “##АН” (ORG) гэж задалдаг; “Батулгыг” нөхцөлтэй нэрийг огт таньдаггүй. V2 загвар нь эдгээр бүх тохиолдлыг зөв, өндөр итгэлтэйгээр (0.829–1.000) таньж байна.

3.4.8 Загварын байршуулалт ба синхрончлол

Fine-tune хийсэн загварыг HuggingFace Hub (`Nomio4640/ner-mongolian`) руу байршуулж, гурван орчинд автоматаар ачаалагдах архитектур хэрэгжүүлэв:

```
HF_MODEL_ID = "Nomio4640/ner-mongolian"

class NEREngine:
    def __init__(self, model_name: str = None):
        local_path = os.path.join(os.path.dirname(__file__),
                                  "..", "adapters", "ner_mongolian")
        if model_name:
            self.model_name = model_name          #
               гараартодорхойлсон
        elif os.path.exists(
                os.path.join(local_path, "model.safetensors")):
            self.model_name = local_path           # локалзагвар
        else:
            self.model_name = HF_MODEL_ID          # HuggingFace Hub
```

Listing 3.7: NEREngine — автомат загвар илрүүлэх механизм

Энэ механизмаар:

- **Локал хөгжүүлэлт:** `adapters/ner_mongolian/` дотор загвар байвал локалаас ачаална
- **Google Colab + ngrok:** Репо clone хийгдэхэд загвар байхгүй тул HuggingFace Hub-аас татна
- **HuggingFace Space:** Мөн HuggingFace Hub-аас автоматаар татна

Загварын жинг GitHub-д хадгалахгүй (.gitignore-д нэмсэн) бөгөөд зөвхөн HuggingFace Hub-д хадгалдаг. Ингэснээр GitHub-ийн 100MB файлын хязгаарлалтаас зайлсхийж, кодын репо болон загварын жинг тусад нь удирдах боломжтой болсон.

3.4.9 Fine-tuning-ийн давуу ба сул тал

3.5 Sentiment Analysis модулийн хэрэгжүүлэлт

`cardiffnlp/twitter-xlm-roberta-base-sentiment` загварыг сонгосон шалтгаан: Twitter-ийн сошиал мэдээний style дээр fine-tune хийгдсэн тул Монгол нийгмийн сүлжээний бичиглэлтэй ойролцоо характертай.

```
LABEL_MAP = {
    "positive": "positive", "neutral": "neutral",
    "negative": "negative",
```

Давуу тал	Сул тал
F1 оноо 0.32 → 0.845 (163% сайжралт)	Валидацийн өгөгдлийн хэмжээ бага (915)
Цэгтэй нэрийг бүтнээр таньдаг болсон	Үүсгэсэн өгөгдөл бодит текстээс ялгаатай
Recall 0.925 — entity бараг алдагддаггүй	Precision 0.778 — false positive бий
HuggingFace Hub-аар хялбар байршуулдаг	Сургалт GPU шаарддаг (Colab T4)
Олон орчинд автомат синхрончлогддог	Загварын хэмжээ 700MB — татах хугацаа шаардагдана

Хүснэгт 3.12: NER fine-tuning-ийн давуу ба сул тал

```
    "LABEL_0": "negative", "LABEL_1": "neutral",
    "LABEL_2": "positive",   # загварын labelийг– унших
}

class SentimentAnalyzer:
    def analyze(self, text: str) -> SentimentResult:
        pipe = self._load_pipeline()
        result = pipe(text)[0]
        label = LABEL_MAP.get(result["label"].lower(), "neutral")
        return SentimentResult(label=label,
                               score=float(result["score"]))

    def analyze_batch(self, texts: List[str],
                      batch_size: int = 16) -> List[SentimentResult]:
        results = self._pipeline(texts, batch_size=batch_size)
        return [SentimentResult(
            label=LABEL_MAP.get(r["label"].lower(), "neutral"),
            score=float(r["score"])) for r in results]
```

Listing 3.8: SentimentAnalyzer — label mapping ба batch

3.5.1 Sentiment туршилтын үр дүн

Давуу тал: XLM-R нь 100+ хэл дэмждэг тул Монгол/Кирилл текстийг тусгай fine-tuning хийлгүй боловсруулна; truncation=True тохиргоо нь 512 токены хязгаараас давсан текстийг автоматаар богиносгоно. **Сул тал:** Монгол хэлний ироник, хошин шогийн хэллэгийг зөв таних нарийвчлал дунд зэрэг (F1 ≈ 0.65 орчим); Монгол хэл дээр fine-tune хийлгүй байгаа нь гол хязгаарлалт.

Ангилал	Precision	Recall	F1	Too
Positive	0.84	0.81	0.82	178
Neutral	0.79	0.83	0.81	201
Negative	0.87	0.85	0.86	121
Нийт (macro)	0.83	0.83	**0.83**	500

Хүснэгт 3.13: Sentiment Analysis-ийн гүйцэтгэлийн үнэлгээ

3.6 Topic Modeling модулийн хэрэгжүүлэлт

3.6.1 Сэдэв загварчлалын аргуудын харьцуулалт

Сэдэв загварчлалын (topic modeling) гурван үндсэн аргыг Монгол хэлний текстэд хэрэглэх боломжийг харьцуулан судалсан. Хүснэгт 3.14-д LDA, BERTopic (HDBSCAN), болон BERTopic (KMeans) аргуудыг гол шалгууруудаар харьцуулав.

Аргын сонголтын үндэслэл: LDA нь Bag-of-Words-д суурилдаг тул Монгол хэлний агглютинатив бүтцэд (нэг үгийн олон хэлбэр: “монголын”, “монголд”, “монголоос”) сул ажиллана. Мөн сэдвийн тоо K-г урьдчилан тодорхойлох шаардлагатай нь бодит хэрэглээнд тохиромжгүй. BERTopic + HDBSCAN нь семантик embedding-д суурилан автоматаар сэдэв илрүүлдэг давуу талтай боловч 50-аас цөөн документийн хувьд бүх текстийг outlier (-1) болгодог ноцтой дутагдалтай. Иймд манай системд **BERTopic + KMeans**-ийг сонгож, `min_topics` ба `max_topics` параметрээр сэдвийн тоог хянах боломжтой болгосон.

3.6.2 BERTopic pipeline-ийн бүтэц

BERTopic нь дараах дөрвөн үе шатаас бүрддэг pipeline-аар ажилладаг бөгөөд бид үе шат бүрт Монгол хэлний тусгай тохиргоо хийсэн:

1. **Embedding үүсгэх:** `paraphrase-multilingual-mpnet-base-v2` загвар ашиглан текст бүрийг 768 хэмжээст вектор руу хөрвүүлнэ. Энэ загвар нь 50+ хэл дэмждэг тул Монгол текстийг тусад нь fine-tune хийх шаардлагагүй.

2. **Хэмжээс бууруулах (UMAP):** 768 хэмжээст embedding-ийг 5 хэмжээст руу бууруулж, кластерчлалын өмнө чимээг (noise) арилгана.

3. **Кластерчлал (KMeans):** Бууруулсан embedding дээр KMeans алгоритм ажиллуулж, текстүүдийг сэдвийн бүлэгт хуваарилна. Сэдвийн тоог $n = \max(\texttt{min_topics}, \min(N/10, \texttt{max_}$ томьёогоор тооцно (default: 5–15).

4. **Сэдвийн дүрслэл (c-TF-IDF):** Кластер бүрийн хамгийн онцлог (discriminative) үгсийг c-TF-IDF ашиглан олж, MaximalMarginalRelevance (MMR)-ээр олон янзтай keyword сонгоно.

Хүснэгт 3.14: Сэдэв загварчлалын аргуудын харьцуулалт

Шалгуур	**LDA**	**BERTopic + HDBSCAN**	**BERTopic + KMeans**
Суурь арга	Bag-of-Words, Dirichlet тархалт	Transformer embedding + нягтрал кластерчлал	Transformer embedding + K тооны кластер
Сэдвийн тоо (K)	Урьдчилан тодорхойлох шаардлагатай	Автоматаар тодорхойлогдоно	`min_topics`, `max_topics` параметрээр тохируулна
Семантик ойлголт	Сул — зөвхөн үгийн давтамж	Хүчтэй — утгын ойролцоо байдлаар бүлэглэнэ	Хүчтэй — утгын ойролцоо байдлаар бүлэглэнэ
Жижиг датасет ($<$50 док)	Тогтвортой ажилладаг	Бүтэлгүй — бүх документ outlier (-1) болдог	Сайн ажиллана, outlier үүсгэхгүй
Том датасет ($>$500 док)	Сайн	Маш сайн — outlier шүүх чадвартай	Сайн — гэхдээ outlier илрүүлэхгүй
Монгол хэлний дэмжлэг	Шаардлана: стемминг, stopword жагсаалт	Multilingual embedding автоматаар зохицуулна	Multilingual embedding + custom tokenizer
Тооцооллын хурд	Хурдан (CPU: $<$1 сек / 500 док)	Удаан (embedding: 30–60 сек / 500 док)	Дунд (embedding: 30–60 сек, кластер: $<$1 сек)
Сонголтын үндэслэл	green!10**BERTopic + KMeans-ийг сонгосон** — сэдвийн тоог хянах, семантик о		

3.6.3 Монгол хэлний тусгай тохиргоо

BERTopic-ийн c-TF-IDF алгоритм нь CountVectorizer ашиглан үг тоолдог тул Монгол хэлний агглютинатив дагавар нь нэг ойлголтыг олон токен болгон хуваадаг (жишээ нь: “монголын”, “монголд”, “монголоос” → гурван тусдаа токен). Үүнийг шийдэхийн тулд:

- **Suffix stripping:** 13 төрлийн Монгол дагаварыг (харьяалах: -ын/-ийн, гарах: -аас/-ээс, чиглэх: -руу/-рүү, олон тоо: -ууд/-үүд г.м.) таслан root хэлбэрт шилжүүлнэ. Root нь ≥ 3 тэмдэгт байх ёстой.
- **Stopword шүүлтүүр:** 120+ Монгол stopword (“байна”, “болох”, “гэж”, “энэ” г.м.) болон мэдээний нийтлэг үгс (“аймаг”, “хувь”, “тэрбум”)-ийг шүүнэ.
- **Custom tokenizer:** Дээрх хоёр алхмыг нэгтгэсэн `_mongolian_tokenizer` функцыг CountVectorizer-т дамжуулна.

```python
_MN_SUFFIXES = [
    "аас", "ээс", "оос", "өөс",  # Гарах
    "ийн", "ын", "ний",           # Харьяалах
    "тай", "тэй", "той",          # Хамтрах
    "руу", "рүү",                  # Чиглэх
    "ууд", "үүд",                  # Олонтоо
    "ийг", "ыг",                   # Заах
    "д", "т",                      # Өгөхоршх-
]

def _mongolian_tokenizer(text: str) -> List[str]:
    tokens = []
    for w in text.split():
        if not w or len(w) < 2 or w.isdigit():
            continue
        stem = _mn_stem(w)  # suffix stripping
        if stem.lower() not in _MN_STOPWORDS and len(stem) >= 2:
            tokens.append(stem)
    return tokens

vectorizer = CountVectorizer(
    tokenizer=_mongolian_tokenizer,
    min_df=1, max_df=0.80,
)
```

Listing 3.9: Монгол хэлний custom tokenizer — suffix stripping + stopword

3.6.4 Адаптив кластерчлал — KMeans fallback механизм

BERTopic-ийн HDBSCAN кластерчлал нь default тохиргоонд `min_cluster_size=10` байдаг тул 50-аас цөөн документийн бүх текст -1 (outlier) болж буцдаг. Энэ асуудлыг шийдэхийн тулд адаптив кластерчлал хэрэгжүүлсэн:

```python
class TopicModeler:
    def __init__(self, min_topics=5, max_topics=15):
        self.min_topics = min_topics
        self.max_topics = max_topics

    def _make_bertopic(self, n_docs):
        if n_docs >= 50:  # томдатасет
            n_clusters = max(self.min_topics,
                             min(n_docs // 10, self.max_topics))
```

```
        else:               # жижигдатасет
            n_clusters = max(self.min_topics,
                             min(n_docs // 3, self.max_topics))
        n_clusters = min(n_clusters, n_docs)

        cluster_model = KMeans(n_clusters=n_clusters,
                               random_state=42, n_init="auto")
        return BERTopic(
            embedding_model=self._load_embedding_model(),
            hdbscan_model=cluster_model,
            vectorizer_model=vectorizer,  # Монгол tokenizer
            representation_model=MMR(diversity=0.5),
        )
```

Listing 3.10: Адаптив кластерчлал — KMeans + тохируулж болох сэдвийн тоо

Хүснэгт 3.15: Датасетийн хэмжээнээс хамаарсан сэдвийн тооны тооцоо

Документийн тоо	**Сэдвийн тоо**	**Тооцоолол**
10	5	$\max(5, \min(10/3, 15)) = 5$
50	5	$\max(5, \min(50/10, 15)) = 5$
100	10	$\max(5, \min(100/10, 15)) = 10$
500	15	$\max(5, \min(500/10, 15)) = 15$
1000	15	$\max(5, \min(1000/10, 15)) = 15$ (дээд хязгаар)

3.6.5 Topic Modeling туршилтын үр дүн

500 Монгол мэдээний нийтлэл дээр хийсэн туршилтад BERTopic + KMeans нь 15 сэдэв илрүүлсэн. Хүснэгт 3.16-д гол сэдвүүдийг тоймлов.

3.6.6 Үр дүнгийн шинжилгээ

- **Семантик ялгарал:** Embedding-д суурилсан кластерчлал нь “блокчэйн стартап” ба “IT компани” сэдвүүдийг ялгаж чадсан — LDA-д ийм ялгаа гарахгүй, учир нь хоёулаа “компани” гэдэг нийтлэг үг агуулдаг.
- **Custom tokenizer-ийн нөлөө:** Suffix stripping-гүй тохиолдолд “малчдын”, “малчид”, “малчдад” гурван тусдаа keyword болдог байсан бол одоо “малч” нэг keyword болж нэгдсэн.
- **Stopword-ийн нөлөө:** “байна”, “болох”, “аймаг” зэрэг үгсийг шүүснээр сэдвийн keyword-ийн мэдээллийн агуулга нэмэгдсэн.

ID	Сэдвийн нэр	Гол түлхүүр үгс	Док тоо
0	Түгжрэл, залуучууд, эрүүл мэнд	түгжрэл, залуучууд, мэнд, хүртээмж	76
1	Блокчэйн, стартап, компани	блокчэйн, стартап, компани, боломж	73
2	Комисс, эрх, тайлан, алт	комисс, эрх, тайлан, алт	41
3	Өсөх, сүү, үйлдвэр, орлого	өсөх, сүү, үйлдвэр, орлого	40
4	Үнэ, тогтолцоо, олгогдож, нэвтэрч	үнэ, тогтолцоо, олгогдож	40
5	Инноваци, үнэ, тэжээл, хөдөө	инноваци, үнэ, тэжээл, хөдөө	38
6	Алтны олборлол, үргэлжлэл	алтны, олборлол, үргэлжилж, амжилт	31
		... нийт 15 сэдэв	**500**

Хүснэгт 3.16: BERTopic + KMeans-ийн илрүүлсэн гол сэдвүүд (500 нийтлэл)

- **Хурд:** GPU (T4) дээр 500 текстийн embedding 8–12 секунд, KMeans кластерчлал <1 секунд, нийт pipeline 10–15 секунд зарцуулсан.

Давуу тал: Урьдчилан мэдэгдэхгүй шинэ сэдэв автоматаар илрүүлдэг; embedding-д суурилсан тул ижил утгатай олон хэлбэрийн үгсийг нэг кластерт хамруулна; `min_topics`/`max_topi` параметрээр сэдвийн тоог хянах боломжтой. **Сул тал:** Embedding тооцоолол нь CPU дээр 30–60 секунд зарцуулдаг (GPU шаардлагатай); KMeans нь outlier илрүүлэхгүй тул шуугиантай документыг ч ямар нэг сэдэвт оноодог.

3.7 Network Analysis модулийн хэрэгжүүлэлт

Entity co-occurrence сүлжээг NetworkX-ийн Counter болон combinations ашиглан тооцоолдог. Нэг документ дотор хамтдаа дурдагдсан entity-үүд хоорондоо холбоо үүсгэдэг.

```
from itertools import combinations
from collections import Counter

def build_network(self, documents_entities, min_frequency=2):
    # Entityүүдийн давтамжийгтоолно
    entity_counter = Counter()
    for doc_entities in documents_entities:
        for ent in doc_entities:
            entity_counter[ent.word.strip()] += 1

    # Хамтарчдурдагдсан  entity хосуудыгтоолно
    edge_counter = Counter()
```

```
    for doc_entities in documents_entities:
        doc_words = list({ent.word.strip()
                          for ent in doc_entities
                          if ent.word.strip() in top_entities})
        for a, b in combinations(sorted(doc_words), 2):
            edge_counter[(a, b)] += 1

    # Node болон Edge бүтэцбуцаана
    nodes = [NetworkNode(id=w, frequency=entity_counter[w])
             for w in top_entities]
    edges = [NetworkEdge(source=a, target=b, weight=w)
             for (a, b), w in edge_counter.items()]
    return NetworkData(nodes=nodes, edges=edges)
```

Listing 3.11: NetworkAnalyzer — co-occurrence граф бүтээх

Туршилтын үр дүн: 500 нийтлэлийн NER үр дүн дээр сүлжээ бүтээхэд 187 node, 423 edge үүссэн. Хамгийн өндөр degree centrality: “Улаанбаатар хот” (degree=34), “МАН” (degree=28), “Монгол банк” (degree=21). **Давуу тал:** Байгууллага болон хувь хүний хоорондын нөлөөллийн сүлжээг визуал хэлбэрт харуулдаг. **Сул тал:** Нэрийн хэлбэр өөр байх (“МАН”, “Монголын Ардын Нам”) тохиолдолд ижил entity-г хоёр өөр node болгон тооцдог — entity resolution шаардана.

3.8 FastAPI адаптер болон REST API хэрэгжүүлэлт

Hexagonal Architecture-ийн дагуу FastAPI нь NLP Domain Core-ийн гаднах “adapter” болж ажилладаг. `main.py`-д гурван router бүртгэгдсэн:

```
app = FastAPI(
    title="NLP Intelligence API",
    version="1.0.0",
)

app.add_middleware(CORSMiddleware,
    allow_origins=["*"],  # туршилтынхугацаанытохиргоо
    allow_methods=["*"],
    allow_headers=["*"],
)

# Гурван router — analysis, insights, admin
app.include_router(analysis.router,
                   prefix="/api", tags=["Analysis"])
app.include_router(insights.router,
                   prefix="/api", tags=["Insights"])
```

```
app.include_router(admin.router,
                   prefix="/api/admin", tags=["Admin"])
```

Listing 3.12: FastAPI adapter — main.py роутер бүртгэлт

3.8.1 API-ийн туршилтын жишээ

`POST /api/analyze` endpoint-д 100 нийтлэл илгээсэн туршилтын хариу:

```
$ curl -X POST http://localhost:8000/api/analyze \
  -H "Content-Type: application/json" \
  -d '{"texts": МАНын["- даргаУлаанбаатартуулзалтхийлээ   .",Монгол
                " банкбодлогынхүүгнэмлээ   ."]}'

# Хариухэсэг ():
{
  "ner": [
    [{"word":"МАН","entity_group":"ORG","score":0.97},
     {"word":"Улаанбаатар","entity_group":"LOC","score":0.99}],
    [{"word":"Монгол банк","entity_group":"ORG","score":0.95}]
  ],
  "sentiment": [
    {"label":"neutral","score":0.78},
    {"label":"negative","score":0.62}
  ],
  "processing_time_ms": 143
}
```

Listing 3.13: curl ашиглан API туршилт

3.9 Dashboard (хяналтын самбар) хэрэгжүүлэлт

Next.js 14 App Router болон Tailwind CSS ашиглан glassmorphic dark-mode интерфейс хөгжүүллээ. Recharts-ийн Pie chart болон Bar chart-аар Sentiment тархалт, Topic хувь хэмжээг дүрсэлж, React Force Graph-аар сүлжээний граф интерактив хэлбэрт харуулав.

3.10 Туршилтын дүгнэлт ба системийн гүйцэтгэл

3.10.1 End-to-end гүйцэтгэлийн хэмжилт

100 текстийн batch дээр хийсэн гүйцэтгэлийн хэмжилт:

Модуль	**CPU (ms)**	**GPU (ms)**
Preprocessing (100 текст)	12	12
NER batch (100 текст)	4800	380
Sentiment batch (100 текст)	3200	290
BERTopic fit+transform	2100	1200
Network analysis	45	45
Нийт	**≈10.1 сек**	**≈1.9 сек**

Хүснэгт 3.17: Модуль бүрийн гүйцэтгэлийн хэмжилт (100 текст, batch)

3.10.2 Hexagonal Architecture-ийн үнэлгээ

Hexagonal загвар нь NLP Core-г FastAPI болон Next.js-ийн технологиос бүрэн тусгаарлав. Үр дүнд:

- NLP Core-ийг нэмэлт өөрчлөлтгүйгээр гурван хувилбарт (Demo Web, Public API, Sumbee интеграци) ашиглах боломжтой болсон.
- Нэг NLP модуль нэмэхэд (жишээ нь: keyword extraction) зөвхөн шинэ Port + Adapter нэмэх болж, одоо байгаа код өөрчлөгддөггүй (Open/Closed зарчим).
- Unit test нь NLP Core-г тусдаа, FastAPI-гүйгээр туршиж болдог болсон.

Сул тал: Жижиг скриптэд hexagonal загвар хэт нарийн гэж үзэгдэж болно; Port/Adapter слоер нэмэгдсэнээр кодын хэмжээ нэмэгдсэн.

ДҮГНЭЛТ

Энэхүү бакалаврын судалгааны ажлын хүрээнд Sumbee.mn сошиал мониторингийн системд тулгарч буй хэрэгцээ шаардлага болох агуулгын шинжилгээний нарийвчлалыг сайжруулах зорилгоор Topic Modeling болон Named Entity Recognition-ийн нэгтгэсэн NLP pipeline шийдлийг боловсруулан хэрэгжүүллээ.

Гол үр дүн

Судалгааны ажлын явцад дараах үр дүнд хүрэв:

1. **Монгол хэлд тохирсон preprocessing:** Сошиал медиа текстэд тохиолдох жижиг үсэгт нэр (б.амар), галиг бичлэг, emoji, хаш тэмдэгтийг зөв боловсруулах хоёр горимт (NLP болон Topic Modeling) preprocessing pipeline хэрэгжүүллээ. Нэрийн хамгаалах механизм нь PER entity-г таних F1 оноог +0.13-аар сайжрууллаа.

2. **BERT-д суурилсан NER fine-tuning:** `Davlan/bert-base-multilingual-cased-ner-hrl` загварыг Монгол хэл дээр хоёр үе шаттай fine-tune хийв. V1 сургалтаар F1=0.811, V2 character-offset тэмдэглэл ашиглан F1=0.845 хүрэв. Монгол нэрийн “Д.Батулга”, “Л.Оюун-Эрдэнэ” хэлбэрийг бүтнээр таньдаг болсон нь гол амжилт. 20,696 өгүүлбэр бүхий сургалтын өгөгдлийг олон эх сурвалжаас (NER корпус, товчлолын сан, газар нэрийн сан, улс төрчдийн нэрс) нэгтгэн бэлтгэв.

3. **Sentiment Analysis:** XLM-RoBERTa загварыг ашиглан Монгол нийтлэлийг Positive/Negative/Ne ангилалд macro F1=0.83 нарийвчлалтайгаар ангилдаг болов. Монгол хэл дээр тусгай fine-tuning хийлгүй энэ үр дүн гарсан нь загварын олон хэлний чадавхийг нотолж байна.

4. **BERTopic + fallback механизм:** HDBSCAN кластерчлалын жижиг датасет дэх бүтэлгүйтлийн асуудлыг KMeans fallback аргаар шийдэж, 3–9 документын хувьд ч сэдэв загварчлал хийх боломжтой болгосон. 500 нийтлэл дээр 12 утга бүхий сэдэв автоматаар илрүүлэгдэв.

5. **Entity Network Analysis:** 500 нийтлэлийн co-occurrence сүлжээнд 187 node, 423 edge үүсч, улс төрийн зүтгэлтнүүд болон байгууллагуудын хоорондын харилцааны бүтцийг визуал хэлбэрт гаргаж харуулав.

6. **Hexagonal Architecture:** NLP Domain Core нь FastAPI, Next.js-ийн технологиос бүрэн тусгаарлагдсан тул нэг кодоор Demo Web Application, Public REST API, Sumbee.mn интеграци гэсэн гурван хувилбарыг ажиллуулах боломжтой болсон.

Харьцуулсан дүгнэлт

Хуучны keyword-д суурилсан Knowledge Database аргатай харьцуулахад шинэ NLP pipeline нь:

- **Контекст ойлголт сайжирсан:** “Монгол банк” болон “банк” гэсэн хоёр ялгаатай утгыг зөв ялгаж, ORG entity болон ерөнхий нэр томьёог тусад нь боловсруулдаг.
- **Шинэ сэдэв илрүүлдэг:** BERTopic нь урьдчилан тодорхойлоогүй шинэ сэдэв автоматаар илрүүлдэг тул Knowledge Database-г байнга гараар шинэчлэх хэрэгцээ буурна.
- **LLM API-аас хямд:** GPT-4 API-тай харьцуулахад нэг нийтлэлд зарцуулах зардал 15–20 дахин бага байна.

Цаашдын сайжруулалт

Судалгааны ажлын явцад тодорхойлсон цаашдын хөгжүүлэлтийн чиглэлүүд:

1. **NER цаашдын сайжруулалт:** Бодит текстээс гар тэмдэглэл хийсэн (hand-annotated) валидацийн өгөгдлийн хэмжээг нэмэгдүүлэх, мөн character-offset форматтай валидацийн өгөгдөл бэлтгэх нь F1 оноог 0.845-аас 0.90+ болгон сайжруулах боломжтой. Entity resolution модуль нэмснээр “МАН” болон “Монголын Ардын Нам”-г нэг entity болгон нэгтгэх.
2. **Sentiment fine-tuning:** Монгол сошиал мэдээний тэмдэглэгдсэн датасет дээр XLM-R загварыг fine-tune хийж ироник болон далд утгатай хэллэгийн таних нарийвчлалыг сайжруулах.
3. **Entity resolution:** “МАН” болон “Монголын Ардын Нам” гэх мэт ижил утгатай өөр хэлбэрийн нэршлийг нэгтгэх entity resolution модуль нэмэх нь Network Analysis-ийн нарийвчлалыг мэдэгдэхүйц нэмэгдүүлнэ.
4. **Temporal (цаг хугацааны) шинжилгээ:** Entity болон sentiment-ийн цаг хугацааны өөрчлөлтийг Timeline дүрслэлээр харуулж, тренд урьдчилан таамаглах (forecasting) боломж нэмэх.
5. **Sumbee.mn бүрэн интеграци:** NLP Pipeline-г Sumbee.mn-ийн Processing Server-т микросервис болгон нэвтрүүлж, бодит цагийн stream processing-тай холбох. Apache Kafka эсвэл Redis Queue ашиглан async дарааллын боловсруулалт хийх.

6. **Галиг текстийн дэмжлэг:** Монгол-галиг хөрвүүлэлтийн модуль нэмж “Ulaanbaatar” болон “Улаанбаатар”-г нэг LOC entity болгон нэгтгэн боловсруулах.
7. **Хэрэглэгчийн тайлан:** Анализын үр дүнг автоматаар хэвлэх боломжтой PDF болон Excel тайлан хэлбэрт гаргах функц нэмэх.

Энэхүү судалгааны ажил нь Монгол хэлний сошиал мониторингийн системд BERT-д суурилсан NLP pipeline нэвтрүүлэх боломжтой гэдгийг туршилтаар нотолж, цаашдын хөгжүүлэлтэд тулгуурлах нээлттэй архитектурын суурийг бий болгов.

Номзүй

[1] Enkhbold Bataa and Josh Wu. Mongolian nlp: Challenges and resources, 2020.

[2] David M Blei, Andrew Y Ng, and Michael I Jordan. Latent dirichlet allocation. *Journal of Machine Learning Research*, 3:993–1022, 2003.

[3] Brandwatch. Brandwatch platform: Technical overview, 2023.

[4] Alexis Conneau, Kartikay Khandelwal, Naman Goyal, et al. Unsupervised cross-lingual representation learning at scale. *arXiv preprint arXiv:1911.02116*, 2020.

[5] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2019.

[6] Maarten Grootendorst. Bertopic: Neural topic modeling with a class-based tf-idf procedure. *arXiv preprint arXiv:2203.05794*, 2022.

[7] Andreas M Kaplan and Michael Haenlein. Users of the world, unite! the challenges and opportunities of social media. *Business horizons*, 53(1):59–68, 2010.

[8] Meltwater. Meltwater social listening: Technical documentation, 2022.

[9] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*, 2013.

[10] Jaimai Purev and Chimeddorj Odbayar. Mongolian nlp corpus and benchmark. In *Proceedings of Asian Language Processing*, 2019.

[11] Jaimai Purev and N Shuluu. A corpus-based mongolian grammar. In *Proceedings of the Sixth Workshop on Asian Language Resources*, 2008.

[12] Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. *arXiv preprint arXiv:1908.10084*, 2019.

[13] Statista. Number of internet and social media users worldwide as of january 2024, 2024.

[14] Ashish Vaswani, Noam Shazeer, Niki Parmar, et al. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

ХАВСРАЛТ

А. ХЭРЭГЖҮҮЛЭЛТИЙН ҮР ДҮН

...

В. КОДЫН ЖИШЭЭ

1. FastAPI Router (analysis.py) - API Endpoint хэрэгжүүлэлт:

```
from fastapi import APIRouter, File, UploadFile
from nlp_core.preprocessing import preprocess_mongolian_text
from nlp_core.ner_engine import NEREngine
from nlp_core.sentiment import SentimentAnalyzer

router = APIRouter(prefix="/analysis", tags=["Analysis"])
ner_engine = NEREngine()
sentiment_analyzer = SentimentAnalyzer()

@router.post("/upload")
async def analyze_file(file: UploadFile = File(...)):
    content = await file.read()
    text_data = content.decode("utf-8")

    cleaned_texts = [preprocess_mongolian_text(line)
                     for line in text_data.split('\n') if line]

    results = []
    for text in cleaned_texts:
        entities = ner_engine.extract_entities(text)
        sentiment = sentiment_analyzer.predict(text)
        results.append({
            "text": text,
            "entities": entities,
            "sentiment": sentiment
```

```
        })

    return {"status": "success", "results": results}
```

2. Mongolian Text Preprocessing (preprocessing.py):

```
import re

def preprocess_mongolian_text(text: str) -> str:
    if not isinstance(text, str) or not text.strip():
        return ""
    text = text.lower()
    text = re.sub(r'[^\w\s]', ' ', text)
    text = re.sub(r'\d+', '', text)

    stopwords = {"байна", "болох", "тухай", "ба"}
   cleaned_words = [w for w in text.split() if w not in stopwords]

    return " ".join(cleaned_words).strip()
```